Especificación del solver:

`highOrderElectroActivationFoamImplicitPETScDistributed`

Un solver monodominio de alto orden, implícito, Jacobian-free Newton–Krylov en OpenFOAM con back-end de álgebra lineal conmutable Eigen/PETSc y álgebra lineal en memoria distribuida

Pablo

18 de octubre de 2026

Resumen

Este reporte especifica los algoritmos implementados en el solver de OpenFOAM:

`highOrderElectroActivationFoamImplicitPETSc`.

El solver avanza las ecuaciones monodominio de la electrofisiología cardíaca acopladas al modelo celular iónico ten Tusscher–Noble–Noble–Panfilov (TNNP, 2004). El solver tiene varias características:

- El operador de difusión se discretiza con el marco de volúmenes finitos centrados en celda de OpenFOAM, opcionalmente mejorado a una reconstrucción de alto orden Local Regression Estimators (LRE);
- La fuente iónica se evalúa ya sea en los centros de celda o en los puntos de cuadratura de la celda (necesario para alto orden);
- Existen diferentes formas para integrar las ecuaciones ODE de reacción:
  (i) en cada punto de cuadratura utilizado para evaluar la fuente iónica;
  (ii) únicamente en los centros y luego reconstruirlos donde sea necesario evaluar la fuente iónica;
  (iii) un híbrido consciente del frente (*front-aware*) que hace (i) solo en el frente propagante y (ii) en el resto (Sección 5);
- La integración temporal sigue un esquema θ (backward Euler o Crank–Nicolson) o un BDF1–4 con arranque ESDIRK3 (desde 2026-10, Sección 13);
- el ODE iónico por celda usa un integrador propio (`RKF45` adaptativo por defecto, `RK4` o `Euler`) con tolerancias configurables, manejado por un polinomio en el tiempo de V_{m} a lo largo del paso;
- El operador de masa puede ser `lumped` o de alto orden `consistent`;
- el sistema no lineal acoplado resultante puede resolverse por iteración de punto fijo de Picard, una linealización diagonal de I_{ion}, o un método Jacobian-Free Newton–Krylov (JFNK) con búsqueda de línea (*line search*) de retroceso de Armijo, valor inicial por extrapolación lineal y un *clamp* de bracket físico sobre la entrada al ODE iónico;
- el sistema lineal de las ramas ensambladas (Picard / diagonalIion) y el sistema interno de JFNK se resuelven con un back-end *conmutable* vía la clave `linearSolverBackend`: o bien las factorizaciones dispersas de Eigen (`SparseLU`/`BiCGSTAB`) y un GMRES reiniciado propio con Gram–Schmidt modificado, o bien los solvers KSP/PC de PETSc (incluido un GMRES sobre una `MatShell` matrix-free para JFNK). El ensamblaje y el almacenamiento de matrices usan siempre `Eigen::SparseMatrix`;
- los tiempos de activación se registran por interpolación lineal a través del primer cruce de umbral y los puntos del benchmark se muestrean por ponderación de distancia inversa.

Todos los parámetros algorítmicos se exponen a través de los diccionarios `constant/spatialIntegrationPropertie` y `constant/timeIntegrationProperties`, y el driver de Python `run_convergence.py` genera estos diccionarios a partir de un único archivo de configuración. Este solver integra ambos back-ends de álgebra lineal (Eigen y PETSc) en un mismo ejecutable, seleccionables en tiempo de ejecución con `linearSolverBackend` / `jfnkLinearSolverBackend` (Sección 7). Existe además un solver hermano más simple, `highOrderElectroActivationFoamImplicit`, con back-end solo Eigen y un esquema semi-implícito de una sola iteración de Picard (Sección E para las diferencias de rendimiento).

Esta variante `...Distributed` hace además *genuinamente distribuida* el álgebra lineal: la malla se descompone con `decomposePar`, los operadores se ensamblan con filas locales y columnas globales y se convierten en matrices `MPIAIJ` de PETSc sobre `PETSC_COMM_WORLD`, y todo producto matriz–vector pasa por `MatMult` en lugar de Eigen. Los algoritmos de las Secciones 5–7 no cambian; la Sección D describe qué exige la descomposición, cómo se verifica un operador paralelo, y la inconsistencia que se medía en la selección de stencils de la librería de alto orden (Sección D.5).

Estado a 2026-10-18. Desde octubre el código que este solver comparte con el solver MMS vive en `src/monodomainCommon/`, y entraron los BDF, el driver de V_{m} por historia, un criterio de parada no lineal común, el precondicionado por derecha del KSP, la historia de V_{m} en los puntos de Gauss y el arreglo del piso temporal del híbrido. La Sección 13 reúne esos cambios y los resultados nuevos (E02, E35, E36 y M02); las secciones anteriores están corregidas donde quedaban falsas. Las tablas de resultados de la Sección 14 son **anteriores** a esos cambios.

Índice

1. Notación **4**

2. Ecuaciones de gobierno **4**
2.1. PDE monodominio . . . 4
2.2. ODE iónico . . . 5
2.3. Ecuación MMS manufacturada usada para transferir la convergencia . . . 5
2.4. Estímulo externo . . . 5

3. Discretización temporal: el esquema θ **6**

4. Discretización espacial **6**
4.1. Laplaciano FV ortogonal estándar . . . 6
4.2. Difusión LRE de alto orden . . . 6
4.3. Matrices de masa . . . 7
4.4. Reconstrucción de alto orden en los puntos de cuadratura iónicos . . . 7
4.5. Promediado de alto orden de I_{ion} a celda . . . 7

5. Integración de states en los puntos de cuadratura iónicos **7**
5.1. Dos modos . . . 7
5.2. Híbrido consciente del frente (front-aware) . . . 8
5.3. Limitador de la reconstrucción de states . . . 8
5.4. Claves de diccionario . . . 9

6. Solver no lineal **9**
6.1. Residuos compartidos y tests de parada . . . 9
6.2. Método de punto fijo de Picard . . . 10
6.3. Linealización diagonal de I_{ion} . . . 10
6.4. Jacobian-Free Newton–Krylov . . . 10
6.5. Krylov interno de JFNK . . . 11
6.6. Búsqueda de línea de retroceso de Armijo . . . 11
6.7. Valor inicial, clamp del ODE y rollback . . . 12

7. Solver lineal: back-ends Eigen y PETSc **12**

8. Integración del ODE iónico **12**

9. Protección numérica del TNNP **13**

10. Seguimiento de la activación y benchmarks **14**

11. Diagrama de flujo del solver **14**

12. Driver manufacturado: `run_convergence.py` **16**
12.1. Geometría y barrido de parámetros . . . 16
12.2. Método no lineal y controles de convergencia . . . 17
12.3. Controles de JFNK y de diagonal-Iion . . . 18
12.4. Solver lineal del PDE . . . 19
12.5. Integrador del ODE de states . . . 19
12.6. Parámetros LRE de alto orden por nivel-p . . . 20
12.7. Diagnósticos y control de corrida . . . 20

13. Cambios de octubre de 2026 y resultados nuevos **20**
13.1. Código común con el solver MMS . . . 20
13.2. BDF1–4 con arranque ESDIRK3 . . . 21
13.3. El driver de V_{m} de los ODEs . . . 21
13.4. La historia de V_{m} en los puntos de Gauss . . . 21
13.5. El piso temporal del híbrido, y su arreglo . . . 22
13.6. Otros cambios . . . 22
13.7. Valores por defecto: solver y harness . . . 23
13.8. Resultados nuevos . . . 23

14. Resultados **23**
14.1. Benchmark 2D estilo Niederer . . . 24
14.1.1. Magnitudes de costo computacional . . . 24
14.1.2. Malla hexaédrica estructurada, sin híbrido (`frontAwareHybrid false`) . . . 25
14.1.3. Malla hexaédrica estructurada, híbrido front-aware (`frontAwareHybrid true`) 27
14.1.4. Malla triangular no estructurada, sin híbrido . . . 28
14.1.5. Malla triangular no estructurada, híbrido front-aware . . . 29
14.2. Benchmark 3D estilo Niederer . . . 32

A. Tabla de dimensiones **33**

B. Mapa de archivos **34**

C. Back-ends de álgebra lineal y solver hermano **34**

D. Paralelización en memoria distribuida **35**
D.1. Disposición de la matriz: filas locales, columnas globales . . . 35
D.2. Qué tiene que cruzar una frontera de procesador . . . 36
D.3. Reducciones globales . . . 37
D.4. Cómo se verifica un operador paralelo . . . 37
D.5. Dependencia de la partición en los stencils MLS . . . 37
D.6. Casos de verificación: qué se corrió y dónde está . . . 39

E. Optimizaciones de rendimiento **40**
E.1. Reutilización de la matriz y la factorización (Fase B) . . . 41
E.2. Precondicionador de difusión para el JFNK (A3) . . . 41
E.3. Integración del ODE iónico en paralelo (C1) . . . 41
E.4. Precondicionadores AMG (C3) . . . 41

F. Experimentos de rendimiento **42**
F.1. Fase B: idéntico y más rápido en Picard . . . 42
F.2. A3: precondicionador de difusión en el JFNK . . . 42
F.3. C1: paralelización del ODE (no ayuda en este problema) . . . 43
F.4. Threading interno de PETSc/BLAS (contraproducente en Picard) . . . 43
F.5. Solver lineal: AMG vs. ILU (el solve no es el cuello) . . . 43
F.6. Síntesis y recomendaciones . . . 44

1. Notación

Símbolo	Significado	Unidades
$\Omega \subset \mathbb{R}^d$	Dominio espacial ($d \in \{2,3\}$)	m
$\partial\Omega$	Frontera aislada (Neumann de flujo nulo)	—
$t \in (0,T]$	Tiempo	s
$V_{\mathrm{m}}(\mathbf{x},t)$	Voltaje transmembrana	V
$\mathbf{s}(\mathbf{x},t) \in \mathbb{R}^{17}$	States iónicos del TNNP	mixtas
$I_{\mathrm{ion}}(V_{\mathrm{m}},\mathbf{s})$	Densidad de corriente iónica total	$\mathrm{V\,s^{-1}}$
$I_{\mathrm{ext}}(\mathbf{x},t)$	Densidad de corriente de estímulo externo	$\mathrm{A\,m^{-3}}$
χ	Razón superficie-a-volumen de membrana	$\mathrm{m^{-1}}$
C_m	Capacitancia de membrana por unidad de área	$\mathrm{F\,m^{-2}}$
$\mathbf{D}(\mathbf{x})$	Tensor de conductividad de volumen	$\mathrm{S\,m^{-1}}$
$\theta \in \{0{,}5, 1\}$	Parámetro del avance temporal (CN o BE)	—
Δt	Longitud del paso de tiempo externo	s
N_c	Número de celdas de malla	—
$\mathsf{V}^n \in \mathbb{R}^{N_c}$	Vector de V_{m} promediado en celda en t^n	V
$\mathbf{M},\mathbf{K},\mathbf{L}$	Matriz de masa, de rigidez, de difusión escalada	—
$\mathbf{A},\mathbf{B}$	Operadores implícitos LHS / RHS del esquema θ	—
$\mathsf{F}(\mathsf{x})$	Residuo no lineal discreto a anular	—

2. Ecuaciones de gobierno

2.1. PDE monodominio

El voltaje de volumen se propaga por la ecuación estándar monodominio de reacción–difusión [1, 2]. En notación física de densidad de corriente puede escribirse

$$\chi\, C_m \frac{\partial V_{\mathrm{m}}}{\partial t} \;=\; \nabla\cdot\big(\mathbf{D}\nabla V_{\mathrm{m}}\big) - \chi C_m\, I_{\mathrm{ion}}(V_{\mathrm{m}},\mathbf{s}) + I_{\mathrm{ext}} \quad \text{en } \Omega\times(0,T], \tag{1}$$

sujeta a fronteras aisladas $\mathbf{n}\cdot(\mathbf{D}\nabla V_{\mathrm{m}}) = 0$ en $\partial\Omega\times(0,T]$, y condición inicial $V_{\mathrm{m}}(\mathbf{x},0) = -84\,\mathrm{mV}$. El tensor de conductividad $\mathbf{D}$ es simétrico definido positivo y ya sea uniforme o leído como un `volTensorField` desde disco. El factor χC_m que multiplica a I_{ion} en (1) refleja la convención usada por la interfaz del modelo iónico: el solver almacena `Iion` como una tasa de voltaje con dimensiones $\mathrm{V\,s^{-1}}$, no como una corriente volumétrica $\mathrm{A\,m^{-3}}$.

Dividiendo (1) por χC_m se obtiene la forma realmente usada dentro del solver,

$$\frac{\partial V_{\mathrm{m}}}{\partial t} \;=\; \mathcal{L}V_{\mathrm{m}} \;+\; s(V_{\mathrm{m}},\mathbf{s}), \qquad \mathcal{L} \;=\; \frac{1}{\chi C_m}\nabla\cdot(\mathbf{D}\nabla\cdot), \quad s \;=\; -I_{\mathrm{ion}} + \frac{I_{\mathrm{ext}}}{\chi C_m}. \tag{2}$$

Esto es exactamente lo que `rhsFromIion` ensambla en el solver.

2.2. ODE iónico

En cada punto de muestreo espacial el vector de states del TNNP $\mathbf{s} \in \mathbb{R}^{17}$ evoluciona según

$$\frac{\mathrm{d}\mathbf{s}}{\mathrm{d}t} \;=\; \mathbf{f}_{\mathrm{TNNP}}\big(V_{\mathrm{m}}, \mathbf{s}; \texttt{tissue}\big), \qquad \mathbf{s}(0) = \mathbf{s}_0, \tag{3}$$

donde `tissue` selecciona `epicardialCells`, `mCells` o `endocardialCells`. Los 17 states rastreados son

$$\mathbf{s} = (V,\ K_i,\ Na_i,\ Ca_i,\ Xr_1,\ Xr_2,\ Xs,\ m,\ h,\ j,\ d,\ f,\ fCa,\ s,\ r,\ g,\ Ca_{SR}).$$

La corriente iónica total devuelta al PDE es la `Iion_cm` algebraica del TNNP [1].

2.3. Ecuación MMS manufacturada usada para transferir la convergencia

El driver de convergencia en `tutorials_electro/highOrderManufacturedFDAImplicitJfNK` resuelve un problema manufacturado con la misma estructura numérica que el solver de electro-activación: PDE implícito tipo monodominio, ODEs locales de states, fuente iónica no lineal, reconstrucción LRE opcional para V_{m} y I_{ion}, y acoplamiento no lineal seleccionable `Picard`/`diagonalIion`/`JFNK`. Los campos analíticos son

$$V_{\mathrm{m}}^{\star}(\mathbf{x}, t) = \sqrt{1+t}\, F(\mathbf{x}), \tag{4}$$
$$u_1^{\star}(\mathbf{x}, t) = (1+t)\, G(\mathbf{x}) + V_{\mathrm{m}}^{\star}(\mathbf{x}, t), \tag{5}$$
$$u_2^{\star}(\mathbf{x}, t) = \frac{1}{(1+t)\sqrt{G(\mathbf{x})}}, \tag{6}$$
$$u_3^{\star}(\mathbf{x}, t) = 0, \tag{7}$$

con, por ejemplo, $F = \cos(\pi x)\cos(2\pi y)$ y $G = 1 + x\,y^2$ en 2D. En 3D, $F = \cos(\pi x)\cos(2\pi y)\cos(3\pi z)$ y $G = 1 + x\,y^2 z^3$. Las ecuaciones de states son

$$\dot{u}_1 = (u_1 + u_3 - V_{\mathrm{m}})^2 u_2^2 + \frac{1}{2}(u_1 + u_3 - V_{\mathrm{m}}) u_2^2 (V_{\mathrm{m}} - u_3), \tag{8}$$
$$\dot{u}_2 = -(u_1 + u_3 - V_{\mathrm{m}}) u_2^3, \tag{9}$$
$$\dot{u}_3 = 0. \tag{10}$$

La fuente iónica manufacturada es

$$I_{\mathrm{ion}}^{\mathrm{MMS}}(V_{\mathrm{m}}, \mathbf{u}) = -\frac{1}{2}(u_1 + u_3 - V_{\mathrm{m}}) u_2^2 (V_{\mathrm{m}} - u_3) + \frac{\beta}{\chi C_m}(V_{\mathrm{m}} - u_3), \tag{11}$$

y la fuente del PDE usada por el solver manufacturado es $-I_{\mathrm{ion}}^{\mathrm{MMS}}$. El coeficiente β es el autovalor espacial del campo trigonométrico manufacturado bajo el tensor de conductividad constante:

$$\beta = \begin{cases} -\pi^2 D_{xx}, & d = 1, \\ -\pi^2 (D_{xx} + 4 D_{yy}), & d = 2, \\ -\pi^2 (D_{xx} + 4 D_{yy} + 9 D_{zz}), & d = 3. \end{cases}$$

Así, el caso MMS no busca reproducir la fisiología del TNNP; es un problema controlado con solución exacta conocida que ejercita los mismos operadores discretos y caminos de solución no lineal usados por este solver.

2.4. Estímulo externo

La corriente externa es un escalón ventaneado en tiempo y en región:

$$I_{\mathrm{ext}}(\mathbf{x}, t) \;=\; \begin{cases} I_0 & \mathbf{x} \in \Omega_{\mathrm{stim}}^{(b)},\ \ t \in [t_0^{(b)},\, t_0^{(b)} + \tau],\ \ b = 1, \dots, B, \\ 0 & \text{en otro caso}, \end{cases} \tag{12}$$

con $I_0 = 50\,000\ \mathrm{A\,m^{-3}}$ y $\tau = 2\,\mathrm{ms}$ como valores por defecto del benchmark de Niederer.

3. Discretización temporal: el esquema θ

El tiempo se avanza de t^n a $t^{n+1} = t^n + \Delta t$ por

$$\frac{\mathsf{V}^{n+1} - \mathsf{V}^n}{\Delta t} \;=\; \theta\,\mathbf{L}\,\mathsf{V}^{n+1} + (1-\theta)\,\mathbf{L}\,\mathsf{V}^n + \theta\,\mathbf{s}^{n+1} + (1-\theta)\,\mathbf{s}^n. \tag{13}$$

`implicitScheme`	θ	Esquema
`backwardEuler`	1	Primer orden, L-estable
`crankNicolson`	1/2	Segundo orden, A-estable

Tras multiplicar por Δt^{-1} la implementación forma

$$\left[\mathbf{M}/\Delta t \;-\; \theta\,\mathbf{L}\right]\mathsf{V}^{n+1} \;=\; \left[\mathbf{M}/\Delta t \;+\; (1-\theta)\,\mathbf{L}\right]\mathsf{V}^n \;+\; \theta\,\mathbf{s}^{n+1} \;+\; (1-\theta)\,\mathbf{s}^n, \tag{14}$$

con los dos operadores Eigen `SparseMatrix<scalar,RowMajor>`

$$\mathbf{A} \;=\; \mathbf{M}/\Delta t \;-\; \theta\,\mathbf{L}, \qquad \mathbf{B} \;=\; \mathbf{M}/\Delta t \;+\; (1-\theta)\,\mathbf{L}. \tag{15}$$

$\mathbf{A}$ y $\mathbf{B}$ se reconstruyen en cada paso de tiempo copiando $\mathbf{M}$, escalando por Δt^{-1} y sumando $\pm\theta\mathbf{L}$. Cuando $\theta = 1$ (BE) la mitad explícita de la difusión se anula y $\mathbf{B} = \mathbf{M}/\Delta t$. La difusión escalada es $\mathbf{L} = \mathbf{K}/(\chi C_m)$.

BDF1–4 (2026-10). `implicitScheme` acepta además `BDF1` (idéntico a `backwardEuler` bit a bit), `BDF2`, `BDF3` y `BDF4`. Terminan en el mismo par de operadores, con $a_0/\Delta t$ en lugar de $1/\Delta t$ delante de $\mathbf{M}$ y la historia sumada como un solo campo; los pasos que un BDF-k no puede dar solo se toman con ESDIRK3. Ver Sección 13.2. A diferencia de lo dicho arriba, $\mathbf{A}$ ya no se reconstruye en cada paso: se reutiliza mientras Δt y el esquema no cambian (Sección E.1).

4. Discretización espacial

4.1. Laplaciano FV ortogonal estándar

`assembleStandardOrthogonalStiffnessMatrix` computa el Laplaciano clásico de 5/7 puntos. Por cada cara interna f con área de cara $\mathbf{S}_f$,

$$a_f \;=\; \frac{|\mathbf{S}_f|\,\left(\mathbf{n}_f \cdot \mathbf{D}_f \cdot \hat{\mathbf{d}}_f\right)}{|\mathbf{d}_f|}, \quad \mathbf{D}_f = \tfrac{1}{2}\left(\mathbf{D}_{\text{own}} + \mathbf{D}_{\text{nei}}\right), \tag{16}$$

aportando $-a_f/V_{\text{own}}$ a $\mathbf{K}[\text{own},\text{own}]$, $+a_f/V_{\text{own}}$ a $\mathbf{K}[\text{own},\text{nei}]$, y el par transpuesto a la fila del vecino. Los patches `zeroGradient`/`empty` aportan flujo nulo. Los patches `fixedValue` aportan solo a la diagonal del dueño (owner).

4.2. Difusión LRE de alto orden

Cuando `useHighOrder_Vm = true` y $d > 1$, `assembleHighOrderStiffnessMatrix` integra el flujo en una cuadratura de cara con gradientes reconstruidos:

$$\Phi_f \;=\; \sum_{q\in\mathcal{Q}_f} w_q\,|\mathbf{S}_f|\,\mathbf{n}_f \cdot \mathbf{D}_f \cdot \Big(\sum_{j\in\mathcal{S}_f} \mathbf{g}_{f,q,j}\,\mathsf{V}_j\Big), \tag{17}$$

donde $\mathcal{S}_f$ es el stencil de cara de la LRE ($\sim$ 15–50 celdas) y $\mathbf{g}_{f,q,j}$ = `QRGradFaceGPCoeffs`. La contribución de cada V_j se suma a la fila `own` con peso $+\Phi_{f,j}/V_{\text{own}}$ y a la fila `nei` con peso $-\Phi_{f,j}/V_{\text{nei}}$, dando un Laplaciano semi-definido negativo.

4.3. Matrices de masa

Dos opciones:

Lumped (`massMatrix lumped`). `assembleDiagonalMassMatrix` escribe $\mathbf{M} = I$.

Consistente HO (`massMatrix consistent`). `assembleConsistentMassMatrixHO` llena

$$\mathbf{M}[i,j] \;=\; \sum_{q\in\mathcal{Q}_i} \frac{w_q}{\sum_{q'} w_{q'}}\,\phi_j(\mathbf{x}_q), \quad \phi_j(\mathbf{x}_q) = \delta_{ij} + \mathbf{g}_{i,j}\cdot\mathbf{d}_q + \tfrac{1}{2}\,\mathbf{H}_{i,j} : (\mathbf{d}_q\otimes\mathbf{d}_q) + \tfrac{1}{6}\,\mathbf{T}^{(3)}_{i,j} \vdots (\mathbf{d}_q\otimes\mathbf{d}_q\otimes\mathbf{d}_q), \tag{18}$$

con $\mathbf{d}_q = \mathbf{x}_q - \mathbf{C}_i$. Una salvaguarda degrada silenciosamente a la forma lumped cuando `useHighOrder_Vm = false` o $d \le 1$.

4.4. Reconstrucción de alto orden en los puntos de cuadratura iónicos

Si `useHighOrder_Iion = true`, el ODE iónico se integra en los puntos de cuadratura de celda. Vm se reconstruye ahí por la misma expansión de Taylor que en (18):

$$V_{\mathrm{m}}(\mathbf{x}_q) \;\approx\; V_{\mathrm{m},c} + (\nabla V_{\mathrm{m}})_c\cdot\mathbf{d}_q + \tfrac{1}{2}\,\mathbf{H}_c : (\mathbf{d}_q\otimes\mathbf{d}_q) + \tfrac{1}{6}\,\mathbf{T}^{(3)}_c \vdots (\mathbf{d}_q\otimes\mathbf{d}_q\otimes\mathbf{d}_q), \tag{19}$$

implementado en `reconstructVmAtIonIntegrationPoints`.

4.5. Promediado de alto orden de I_{ion} a celda

`averageIonIntegrationPointsToCells` contrae el I_{ion} por punto de cuadratura de vuelta a celdas:

$$I_{\mathrm{ion},i} \;=\; \frac{\sum_{q\in\mathcal{Q}_i} w_q\, I_{\mathrm{ion},q}}{\sum_{q\in\mathcal{Q}_i} w_q}. \tag{20}$$

5. Integración de states en los puntos de cuadratura iónicos

La cuadratura de alto orden (19) evalúa $I_{\mathrm{ion}}(V_{\mathrm{m}}, \mathbf{s})$ en los puntos de cuadratura de celda $\mathbf{x}_q$, así que también necesita ahí el vector de states iónicos $\mathbf{s}$. *Cómo* llegan los states a los puntos de cuadratura lo gobierna `stateIntegrationMode` y, ortogonalmente, `frontAwareHybrid`. Esta elección resulta decisiva para un frente cardíaco propagante, y esta sección documenta tanto el algoritmo como la razón por la que importa.

5.1. Dos modos

Sea $\mathcal{Q}_i = \{\mathbf{x}_q\}$ el conjunto de cuadratura de la celda Ω_i con pesos w_q, y sea $\mathbf{s}_c$ un state centrado en celda.

`gaussPointODE` **(heredado).** El ODE rígido (3) se integra *independientemente en cada punto de cuadratura*, manejado por el potencial reconstruido localmente:

$$\mathbf{s}_q^{n+1} = \mathrm{ODEsolve}\big(\mathbf{s}_q^n,\, V_{\mathrm{m}}(\mathbf{x}_q);\, \Delta t\big), \qquad I_{\mathrm{ion},q} = \phi\big(V_{\mathrm{m}}(\mathbf{x}_q), \mathbf{s}_q^{n+1}\big). \tag{21}$$

Costo: N_q integraciones rígidas por celda.

`cellCentredReconstruct`**.** El ODE se integra *una vez por celda* en el centro de celda, y los states centrados en celda resultantes se reconstruyen a los puntos de cuadratura por una LRE dedicada (`LREInterp_states`, orden fijado por `highOrderCoeffs/LRECoeffs_states`):

$$\mathbf{s}_c^{n+1} = \mathrm{ODEsolve}\big(\mathbf{s}_c^n,\, \overline{V}_{m,c};\, \Delta t\big), \quad \widehat{\mathbf{s}}(\mathbf{x}_q) = \mathcal{R}^{(p)}\big[\{\mathbf{s}_{c'}^{n+1}\}\big](\mathbf{x}_q), \quad I_{\mathrm{ion},q} = \phi\big(V_{\mathrm{m}}(\mathbf{x}_q), \widehat{\mathbf{s}}(\mathbf{x}_q)\big). \tag{22}$$

Costo: una integración rígida por celda — un factor N_q más barato, lo cual para las costosas integraciones TNNP es la motivación. Los dos modos difieren solo en el argumento del mapa no lineal $V_{\mathrm{m}} \mapsto \mathbf{s}$: puntual $V_{\mathrm{m}}(\mathbf{x}_q)$ en (21), versus el promedio de celda $\overline{V}_{m,c}$ en (22).

5.2. Híbrido consciente del frente (front-aware)

`frontAwareHybrid` (por defecto `false`, ortogonal a `stateIntegrationMode`) combina los dos modos: resuelve el ODE en los puntos de Gauss en la banda delgada de celdas de *frente* y en el centro de celda (con reconstrucción) en el resto. Una celda i es de *frente* cuando su rango sub-celda de V_m excede `hybridFrontVmRange` o está estimulada,

$$\text{frente}_i \iff \Big(\max_q V_\mathrm{m}(\mathbf{x}_q) - \min_q V_\mathrm{m}(\mathbf{x}_q)\Big) > \texttt{hybridFrontVmRange} \quad \vee \quad (I_\mathrm{ext})_i \neq 0. \tag{23}$$

Los states del frente se llevan en los puntos de Gauss (como en `gaussPointODE`); **desde 2026-10-16 las celdas suaves llevan su propio state de celda**, `hybridCellStates`, como en CC puro. Antes ese state de celda se rehacía en cada paso como el promedio por cuadratura de los states de Gauss, y eso introducía un piso en la convergencia temporal (Sección 13.5). Por evaluación no lineal la rama es:

(a) celdas de frente: un ODE por punto de Gauss, V_m local (como (21));

(b) celdas suaves: un ODE en el centro de celda desde su state de celda del inicio del paso (el promedio de Gauss sólo se usa una vez, cuando una celda sale del frente);

(c) reconstruir los states de las celdas suaves a sus puntos de Gauss, dejando intactos los states de Gauss del frente;

(d) evaluar I_ion en todos los puntos de Gauss y promediar a la celda.

Congelar por paso de tiempo (evitando un ciclo límite). La clasificación de frente (23) es un *switch discreto dependiente de la solución*. Si se recalcula dentro del bucle no lineal, las celdas cerca del umbral alternan frente↔suave entre iteraciones; como las dos ramas computan I_ion por caminos distintos, el término fuente salta discretamente y el mapa de punto fijo de Picard se vuelve discontinuo, produciendo un *ciclo límite de período 2* no convergente (observado como $\mathrm{iter}_{k+2} \equiv \mathrm{iter}_k$ en `nonlinearResiduals.dat`). La clasificación se computa por lo tanto *una vez por paso de tiempo* desde el V_m^n committeado y se *congela* para todo el solve no lineal, y la banda se dilata `hybridFrontDilation` capas de celdas (vía `mesh.cellCells()`) para que el frente permanezca dentro de la zona resuelta en Gauss a medida que avanza. Congelar es seguro porque el frente se mueve mucho menos que una celda por paso (para el benchmark 2D, $\sim$0,09 mm/ms, es decir $\sim$0,009 celda por paso a $\Delta t = 10^{-4}$ s, $\Delta x = 0{,}5$ mm).

5.3. Limitador de la reconstrucción de states

La reconstrucción de alto orden ($p3$) de los states rígidos del TNNP a través de un frente sub-resuelto sobre-dispara (Gibbs), y un state llevado muy fuera de su rango físico puede desbordar (overflow) el álgebra iónica (observado como un `SIGFPE` de PETSc). `stateReconstructLimiter` (por defecto `false` desde 2026-10-06, en el solver y en el harness; activo, cuando se pide, para `cellCentredReconstruct` y las celdas suaves del híbrido, independiente de `frontAwareHybrid`) aplica una cota tipo Barth–Jespersen: cada valor reconstruido se recorta al rango de la celda y sus vecinas de cara,

$$\widehat{s}(\mathbf{x}_q) \;\leftarrow\; \mathrm{clip}\Big(\widehat{s}(\mathbf{x}_q),\; \min_{c' \in \{i\}\cup\mathcal{N}(i)} s_{c'},\; \max_{c' \in \{i\}\cup\mathcal{N}(i)} s_{c'}\Big). \tag{24}$$

No se crea ningún extremo nuevo, así que los states reconstruidos permanecen dentro del rango físicamente realizado (evolucionado por el ODE) y el álgebra iónica nunca ve una entrada patológica; en regiones suaves la reconstrucción ya cae dentro del rango y el limitador queda inactivo, preservando el orden ahí.

Por qué está apagado. En el frente el recorte al rango de los vecinos baja la reconstrucción a primer orden justamente donde importa, y ninguna corrida de la suite actual lo necesita para terminar. Queda como recurso si un caso desborda; sin él, el híbrido puede caer en `SIGFPE` en configuraciones extremas (se vio al probar una histéresis en la clasificación del frente, que se descartó).

5.4. Claves de diccionario

Clave (en `implicitHOCoeffs`)	Significado	Por defecto
`stateIntegrationMode`	`gaussPointODE` / `cellCentredReconstruct`	`cellCentredReconstr`
`frontAwareHybrid`	ODEs en Gauss en el frente, reconstrucción en el resto	`false`
`hybridFrontVmRange`	rango sub-celda de V_{m} que marca celda de frente [V]	5×10^{-3}
`hybridFrontDilation`	capas de celda agregadas alrededor de la banda de frente	2
`stateReconstructLimiter`	limitador Barth–Jespersen en la reconstrucción	`false`
`stateODEDriver`	polinomio de V_{m} que maneja los ODEs: `history` / `linear`	`history`
`stateODEGaussPointHistory`	la historia también en los caminos de Gauss	`false`

`LRECoeffs_states` (en `highOrderCoeffs`) fija el orden de reconstrucción; es requerido por `cellCentredReconstruct` y por el híbrido.

6. Solver no lineal

En cada paso de tiempo externo la incógnita es el nuevo vector de voltaje $\mathsf{x} = \mathsf{V}^{n+1}$. Para cualquier candidato x el solver primero reconstruye el campo V_{m} correspondiente, luego llama a `evaluateNonlinearFields`. Esa rutina resetea los states iónicos a sus valores al inicio del paso $\mathbf{s}^n$, integra los ODEs locales de t^n a t^{n+1} usando el voltaje candidato, obtiene $I_{\mathrm{ion}}(\mathsf{x})$, y opcionalmente promedia los valores de puntos de cuadratura de vuelta a celdas. Esto da el residuo no lineal

$$\mathsf{F}(\mathsf{x}) \;=\; \mathbf{A}\,\mathsf{x} \;-\; \mathsf{r}\big(I_{\mathrm{ion}}(\mathsf{x})\big), \qquad \mathsf{x} = \mathsf{V}^{n+1}, \tag{25}$$

con

$$\mathsf{r}(I_{\mathrm{ion}}) = \mathbf{B}\,\mathsf{V}^n + \theta\left[-I_{\mathrm{ion}}(\mathsf{x}) + I_{\mathrm{ext}}^{n+1}/(\chi C_m)\right] + (1-\theta)\left[-I_{\mathrm{ion}}^{n} + I_{\mathrm{ext}}^{n}/(\chi C_m)\right]. \tag{26}$$

Los tres métodos no lineales difieren solo en cómo llevan a $\mathsf{F}(\mathsf{x})$ hacia cero.

6.1. Residuos compartidos y tests de parada

Cada método registra los mismos diagnósticos:

$$R_{\mathrm{cpl}} = \frac{\|\mathsf{F}(\mathsf{x}_k)\|_2}{\|\mathsf{x}_k\|_2 + \epsilon}, \tag{27}$$

$$R_{V_{\mathrm{m}}} = \frac{\|\mathsf{V}_k - \mathsf{V}_{k-1}\|_2}{\|\mathsf{V}_{k-1}\|_2 + \epsilon}, \tag{28}$$

$$R_{I_{\mathrm{ion}}} = \frac{\|I_{\mathrm{ion},k} - I_{\mathrm{ion},k-1}\|_2}{\|I_{\mathrm{ion},k-1}\|_2 + \epsilon}, \tag{29}$$

$$R_{\mathbf{s}} = \max_j \frac{\|\mathbf{s}_{j,k} - \mathbf{s}_{j,k-1}\|_2}{\|\mathbf{s}_{j,k-1}\|_2 + \epsilon}. \tag{30}$$

La implementación real usa `relativeL2Difference` para $R_{V_{\mathrm{m}}}$ y $R_{I_{\mathrm{ion}}}$, y `maxStateRelativeL2Difference` para el máximo de states. Una iteración no lineal solo puede parar tras `minNonlinearIterations`; la convergencia luego requiere `nonlinearTolerance`, `nonlinearVmTolerance` y `nonlinearIionTolerance`. El criterio de states `nonlinearStatesTolerance` se incluye solo cuando `nonlinearRequireStatesConvergence=true`.

Un solo test, con salida por piso (2026-10). Los tres métodos y las etapas ESDIRK paran con la misma función compartida, `nonlinearIterationExit`: primero tienen que pasar los incrementos; después, si $R_{\mathrm{cpl}} \le$ `nonlinearTolerance` la iteración convergió, y si R_{cpl} no bajó respecto de la iteración anterior se corta por *piso*: el iterado está tan convergido como el operador permite (el residuo verdadero que deja un Krylov precondicionado, o redondeo). Las salidas por piso se cuentan y se informan en el log. Los incrementos nunca se aflojan.

6.2. Método de punto fijo de Picard

La rama Picard trata la fuente como retrasada (lagged) durante el solve del PDE. En la iteración k los states y la fuente iónica se computan desde x_k, luego el PDE lineal

$$\mathbf{A}\,\mathsf{V}_* = \mathsf{r}\big(I_{\text{ion}}(\mathsf{x}_k)\big) \tag{31}$$

se resuelve con el solver disperso Eigen seleccionado por `linearSolver`. El siguiente candidato se relaja,

$$\mathsf{x}_{k+1} = \mathsf{x}_k + \omega(\mathsf{V}_* - \mathsf{x}_k), \qquad \omega = \texttt{nonlinearRelaxation}. \tag{32}$$

Tras la actualización el solver re-evalúa states y I_{ion} en x_{k+1} y chequea los residuos de arriba. En este solver de electro-activación, a diferencia de un estudio Picard MMS puro, un solve Picard fallido hace rollback (reversión) a menos que `nonlinearAcceptUnconverged=true`.

Ventajas. Picard es simple, barato por iteración no lineal, y usa el mismo solver lineal del PDE que la formulación implícita clásica. Es una línea base útil y a menudo es adecuado cuando Δt es chico o la fuente iónica es débilmente no lineal en un paso de tiempo.

Desventajas. La derivada de la fuente respecto a V_{m} se ignora en la matriz del PDE, así que la convergencia puede ser lenta cerca del upstroke del potencial de acción o para Δt grande. Un residuo chico puede requerir muchas iteraciones, y puede necesitarse sub-relajación para robustez.

6.3. Linealización diagonal de I_{ion}

La rama `diagonalIion` es Picard más una aproximación local por diferencias finitas de la derivada de la fuente. Definir

$$S(V_{\text{m}}) = -I_{\text{ion}}(V_{\text{m}}, \mathbf{s}(V_{\text{m}})) + I_{\text{ext}}/(\chi C_m).$$

El solver aproxima, celda a celda,

$$S_i'(V_{m,k}) \;\approx\; -\frac{I_{\text{ion},i}(V_{m,k} + \varepsilon_d) - I_{\text{ion},i}(V_{m,k})}{\varepsilon_d}, \qquad \varepsilon_d = \texttt{diagonalIionEpsilon}. \tag{33}$$

El residuo linealizado $\mathbf{A}\mathsf{x} - \theta S(\mathsf{x})$ da el sistema

$$\big(\mathbf{A} - \theta\,\mathrm{diag}(S_k')\big)\,\mathsf{x}_{k+1} = \mathsf{r}\big(I_{\text{ion}}(\mathsf{x}_k)\big) - \theta\,\mathrm{diag}(S_k')\,\mathsf{x}_k. \tag{34}$$

La actualización aún puede relajarse con `nonlinearRelaxation`, y se usa la misma lógica de convergencia y rollback que en Picard.

Ventajas. Captura la rigidez local dominante de $I_{\text{ion}}(V_{\text{m}})$ sin construir un Jacobiano completo. A menudo reduce el número de iteraciones no lineales respecto a Picard preservando una matriz escalar dispersa del PDE.

Desventajas. Solo se incluye la sensibilidad diagonal local de celda. Los acoplamientos a través de los states del ODE, el promediado de cuadratura de alto orden y la reconstrucción espacial no se representan en la matriz. El paso de diferencias finitas `diagonalIionEpsilon` debe elegirse por encima de los niveles de ODE/ruido pero suficientemente chico para ser una derivada útil.

6.4. Jacobian-Free Newton–Krylov

JFNK aplica Newton a (25):

$$\mathbf{J}(\mathsf{x}_k)\delta_k = -\mathsf{F}(\mathsf{x}_k), \qquad \mathsf{x}_{k+1} = \mathsf{x}_k + \alpha_k\delta_k,$$

pero nunca ensambla $\mathbf{J}$. GMRES ve solo productos matriz-vector aproximados por

$$\mathbf{J}(\mathsf{x}_k)\,\mathbf{v} \;\approx\; \frac{\mathsf{F}(\mathsf{x}_k + \varepsilon\,\mathbf{v}) - \mathsf{F}(\mathsf{x}_k)}{\varepsilon}, \qquad \varepsilon \;=\; \varepsilon_0\,\frac{1 + \|\mathsf{x}_k\|_2}{\|\mathbf{v}\|_2}, \tag{35}$$

con $\varepsilon_0 =$ `jfnkEpsilon`. Cada evaluación de F vuelve a correr los ODEs y recomputa I_{ion} desde el mismo snapshot de state al inicio del paso. Esto es costoso pero da un residuo no lineal genuinamente acoplado.

6.5. Krylov interno de JFNK

El sistema lineal de Newton $\mathcal{J}_k\delta = -\mathsf{F}_k$ se resuelve con un back-end elegido por `jfnkLinearSolverBackend`:

- **Eigen** — `solveGMRES` implementa GMRES(m) con Gram–Schmidt modificado, usando a lo sumo `jfnkMaxKrylovIterations` vectores base por reinicio y `jfnkMaxRestarts` reinicios.
- **PETSc** — `KSPGMRES` sobre una `MatShell` cuyo producto matriz-vector es la envoltura de diferencias finitas; el objeto KSP es persistente entre pasos.

En ambos casos el test de parada lo controla `jfnkLinearTolerance` y el método es "matrix-free" (sin matriz): el operador lineal es la envoltura (shell) de diferencias finitas de (35), no una matriz dispersa almacenada.

Precondicionador (opcional). Con `jfnkPreconditioner=diffusion` se aplica un precondicionador por la derecha $\mathbf{M}^{-1}$ con $\mathbf{M} = \mathbf{A}$ (la parte lineal, difusiva y SPD, del Jacobiano), factorizado una sola vez como ILU incompleto y reutilizado; cada paso de Krylov cuesta entonces un *triangular solve* barato en lugar de una integración del ODE. El valor por defecto `none` reproduce el camino sin precondicionador. Ver Sección E.2.

Ventajas. JFNK ve la dependencia no lineal completa $V_{\text{m}} \mapsto \mathbf{s}(V_{\text{m}}) \mapsto I_{\text{ion}}(V_{\text{m}}, \mathbf{s})$ y usualmente tiene una convergencia no lineal mucho mejor que Picard en regímenes rígidos. También evita formar explícitamente un Jacobiano denso o complicado.

Desventajas. Cada vector de Krylov requiere una evaluación adicional del residuo, y por ende una nueva integración de ODE en cada celda o punto de cuadratura. El método es sensible a la perturbación de diferencias finitas, a las tolerancias del ODE y a las elecciones de la búsqueda de línea. Sin precondicionador (`jfnkPreconditioner=none`) el número de matvecs — y por lo tanto de integraciones del ODE — puede ser alto; el precondicionador de difusión (Sección 6.5) lo reduce marcadamente. Empíricamente, para el benchmark de este reporte JFNK resulta bastante más costoso que Picard incluso con precondicionador (Sección F).

6.6. Búsqueda de línea de retroceso de Armijo

La actualización cruda de Newton puede aumentar el residuo no lineal cuando el modelo lineal es pobre. Si `jfnkLineSearch=true`, el solver prueba

$$\mathsf{x}_{k+1} = \mathsf{x}_k + \alpha\delta_k, \qquad \alpha = \texttt{nonlinearRelaxation},\ \alpha/2,\ \alpha/4, \ldots$$

hasta que se cumple la condición de descenso suficiente de Armijo:

$$\|\mathsf{F}(\mathsf{x}_k + \alpha\delta_k)\|_2^2 \le (1 - 2c\alpha)\ \|\mathsf{F}(\mathsf{x}_k)\|_2^2\,, \qquad c = \texttt{jfnkArmijoC}. \tag{36}$$

El retroceso para tras `jfnkLineSearchMaxIter` intentos o cuando la longitud de paso alcanza `jfnkLineSearchAlphaMin`. Esto globaliza Newton: los pasos grandes de Newton se aceptan cuando reducen el residuo, en caso contrario se amortiguan.

6.7. Valor inicial, clamp del ODE y rollback

El primer valor inicial para el paso de tiempo es ya sea V^n o la extrapolación lineal

$$\mathsf{x}_0 = \mathsf{V}^n + \frac{\Delta t}{\Delta t_{\text{prev}}} \left(\mathsf{V}^n - \mathsf{V}^{n-1} \right), \tag{37}$$

seleccionado por `jfnkInitGuessOrder`. Se recorta a [`jfnkInitGuessVmMin`, `jfnkInitGuessVmMax`]. El mismo bracket también puede aplicarse a los valores de voltaje pasados al solver del ODE mediante `jfnkClampODEInput`; esto protege al modelo iónico durante las sondas de diferencias finitas de JFNK.

Si el bucle no lineal alcanza su presupuesto de iteraciones sin satisfacer los tests de convergencia, `nonlinearAcceptUnconverged` decide si el último iterado se committea. Con el valor por defecto `false`, el solver hace rollback de V_{m}, I_{ion} y todos los states iónicos al inicio del paso de tiempo. El rollback evita que un solve no lineal malo contamine el resto de la simulación de activación.

7. Solver lineal: back-ends Eigen y PETSc

El ensamblaje y el almacenamiento de las matrices ($\mathbf{M}$, $\mathbf{K}$, $\mathbf{L}$, $\mathbf{A}$) usan *siempre* `Eigen::SparseMatrix`. La resolución del sistema lineal de las ramas ensambladas (Picard / diagonalIion) la maneja el envoltorio `solveSparseSystem`, con un back-end elegido por `linearSolverBackend`:

`linearSolverBackend`	`linearSolver`	Implementación
`Eigen`	`SparseLU`	`Eigen::SparseLU` (LU directo)
`Eigen`	`BiCGSTAB`	`Eigen::BiCGSTAB+IncompleteLUT`
`PETSc`	(mapeado)	`KSP+PC` (p.ej. `gmres+ilu`, `preonly+lu`, `cg+gamg`)

Con el back-end PETSc, la matriz de Eigen se copia a una `Mat` de PETSc y el tipo de KSP/PC se toma de `petscLinearKspType` / `petscLinearPcType` (este último acepta `ilu`, `lu`, `jacobi`, `gamg`, `hypre`, ...). Todos comparten `tolerance` y `maxIterations`.

Precondicionado por derecha (2026-10). GMRES y BiCGStab precondicionan por izquierda por defecto en PETSc, y entonces `tolerance` se aplica al residuo *precondicionado*. Con `ilu` y Δt grande los dos difieren en un factor ~ 4, y el criterio no lineal, que mide el residuo verdadero, nunca se alcanzaba. El KSP ensamblado precondiciona ahora por **derecha** (como ya hacía el KSP shell de JFNK), y `cg` monitorea la norma sin precondicionar. Se fija antes de `KSPSetFromOptions`, así que una opción de PETSc todavía puede cambiarlo.

Reutilización de la factorización. Como $\mathbf{A} = \mathbf{M}/\Delta t - \theta\mathbf{L}$ es constante mientras Δt no cambia, la matriz se ensambla una sola vez y el solver (KSP de PETSc o factorización de Eigen) se conserva durante todo el run, refrescándose sólo cuando hace falta según una política de actualización {Rebuild, ValuesOnly, Reuse}. Esto evita re-factorizar en cada iteración no lineal (Sección E.1).

JFNK. La rama matrix-free de JFNK no instancia estos envoltorios ensamblados; usa su propio Krylov (Sección 6.5) sobre la envoltura del Jacobiano de diferencias finitas, con back-end `jfnkLinearSolverBackend` (Eigen o PETSc).

8. Integración del ODE iónico

Para cada celda o punto de cuadratura de celda, el ODE del TNNP (3) se avanza con un integrador propio del solver seleccionado vía la clave `solver`: `RKF45` adaptativo por defecto [6], `RK4` o `Euler`; cualquier otro nombre es `FatalError`. Cada integración RKF45 arranca desde `initialODEStep` (antes heredaba el último sub-paso de la llamada anterior y la clave no tenía efecto). Dentro del paso, V_{m} no está congelado: lo da el driver de la Sección 13.3. Claves leídas de `constant/timeIntegrationProperties`:

Clave	Significado	Por defecto
`solver`	Integrador ODE concreto	`RKF45`
`absTol`	Tolerancia absoluta por state	10^{-9}
`relTol`	Tolerancia relativa por state	10^{-6}
`maxSteps`	Tope de sub-pasos por llamada externa	10^4
`initialODEStep`	Sub-paso inicial (ms)	10^{-5}

`relTol` maneja el piso de ruido del Jacobiano matrix-free: $\eta_{\mathsf{F}} \sim$ `relTol`. Ajustar `relTol` más fino compra una tolerancia externa alcanzable más baja a costa de más sub-pasos de RKF45 por iteración de Newton.

9. Protección numérica del TNNP

El switch maestro `TNNPNumericalProtection` (por defecto `false`) condiciona una guarda local invocada en cada entrada externa al kernel TNNP embebido (`calculateCurrent`, `solveODE_start`, `solveODE_end`, `derivatives`).

Clave	Tipo	Por defecto
`TNNPNumericalProtection`	Switch	`false`
`TNNPClampGates`	Switch	`true`
`TNNPClampVmForRates`	Switch	`true`
`TNNPLogNumericalProtection`	Switch	`true`
`TNNPConcentrationFloor`	scalar (mM)	10^{-12}
`TNNPVMinForRates`	scalar (mV)	-200
`TNNPVMaxForRates`	scalar (mV)	$+100$
`TNNPVoltageZeroEps`	scalar (mV)	10^{-8}
`TNNPMaxProtectionLogEntries`	label	10^6
`TNNPProtectionLog`	word	`TNNP_numericalProtection_summary.dat`

Cuando la protección está habilitada, las guardas aplicadas en orden son:

1. V **no finito**: `!std::isfinite(V)` $\rightarrow$ resetear a $-86{,}2$ mV.
2. **Clamp de V para las tasas** (si `TNNPClampVmForRates`): recortar a [`TNNPVMinForRates`, `TNNPVMaxForRates`].
3. **Evitación de singularidad en $V = 0$**: empujar V fuera de cero por `TNNPVoltageZeroEps` (preservando el signo) cuando $|V|$ cae por debajo de ese umbral.
4. **Piso de concentración** en K_i, Na_i, Ca_i, Ca_{SR}: entradas no finitas o por debajo del piso se resetean a `TNNPConcentrationFloor`.
5. **Clamp de compuertas** en $Xr_1, Xr_2, Xs, m, h, j, d, f, fCa, s, r, g$ (si `TNNPClampGates`): no finitas a cero, recortar a $[0, 1]$.

La implementación embebida almacena un conteo de correcciones acotado y emite un archivo de resumen compacto al salir; no depende de utilidades externas de logging del modelo iónico.

Por qué queda apagada (2026-10). Encendida, en el estudio temporal E02 el contador llega a su tope de 10^6 y los BDF pierden el orden: con states CC, BDF2/3/4 dan 1.74/2.22/3.54 con la protección y 1.89/2.96/4.60 sin ella, y ninguna corrida la necesitó para terminar. Desde 2026-10-16 el harness de la suite tampoco la enciende (`TNNP_NUMERICAL_PROTECTION = False`).

10. Seguimiento de la activación y benchmarks

Tiempo de activación por celda. El primer cruce de V_{thr} en un paso de tiempo se registra por interpolación lineal:

$$w \;=\; \text{clip}\Big(\frac{V_{\text{thr}} - V^n}{V^{n+1} - V^n},\; 0, 1\Big), \qquad t_{\text{act}} \;=\; t^n + w\,\Delta t, \tag{38}$$

con una bandera por celda que se limpia tras el primer cruce para ignorar cruces posteriores.

Terminación temprana en P8. Cuando `stopAfterPointActivation = true`, se monitorea la celda más cercana a `stopActivationPoint` (P8 en el benchmark de Niederer) y `effectiveEndTime` se fija a $t_{\text{act}}+$ `stopDelayAfterActivation` una vez que su $t_{\text{act}} > 0$.

Muestreo de la diagonal. `sampleIDW` realiza interpolación ponderada por distancia inversa en `nDiagonalSamples` puntos entre `diagonalStart` y `diagonalEnd`, con $k =$ `sampleNeighbours`, $p =$ `samplePower`. Como una muestra de la diagonal promedia sus k celdas más cercanas y las celdas no activadas contribuyen $t_{\text{act}} = 0$, una muestra observada *en vivo* sube desde abajo a medida que el frente llena su vecindario; el valor final (las k vecinas activadas) es el correcto.

Salida de activación en vivo. `writeActivationSamples` se llama una vez antes del bucle de tiempo (para que `diagonalActivationTime.dat` y `P8_activationTime.dat` existan desde $t = 0$ con todo punto pendiente) y de nuevo cada vez que el conteo de celdas activadas aumenta, sobrescribiendo los archivos. La corrida puede así monitorearse a medida que el frente avanza; el driver expone el archivo en vivo en cada directorio de corrida mediante un enlace simbólico que se reemplaza por una copia real al final.

Detección de falla de activación. Cuando `abortOnActivationFailure = true` el solver para temprano (limpiamente, exit 0) si la activación falla, evitando una corrida completa hasta `endTime`. Dos criterios, usando el conteo corriente de celdas activadas N_a y el tiempo $t_\star$ de la última activación nueva:

- **sin ignición:** $N_a = 0$ y $t >$ `activationIgnitionTimeout`;
- **bloqueo de conducción:** $N_a > 0$, el punto de parada aún no se alcanzó, y $t - t_\star >$ `activationStallTime`.

El resultado se registra siempre en `postProcessing/.../activationStatus.dat` como uno de `OK`, `FAILED_NO_IGNITION`, `FAILED_CONDUCTION_BLOCK` o `INCOMPLETE` (con el tiempo final y N_a). El driver lee este archivo y, ante una falla, lo loguea en `activation_status_summary.dat` y continúa con la siguiente configuración en lugar de abortar el barrido.

11. Diagrama de flujo del solver

El pseudo-código de abajo sigue el solver real. El setup compartido es común a todos los métodos no lineales; luego se ejecuta uno de los tres bloques específicos según `nonlinearMethod`.

Algorithm 1 Setup compartido del paso de tiempo y commit/rollback final.

Leer diccionarios y ensamblar (una vez) $\mathbf{M}$, $\mathbf{K}$, $\mathbf{L} = \mathbf{K}/(\chi C_m)$.
Inicializar el estado persistente del solver lineal (KSP de PETSc / factorización de Eigen) y marcar Δt como no ensamblado.
while $t^n < T$ **do**
    Guardar snapshots: V^n, I_{ion}^n, $\mathbf{s}^n$.
    if Δt cambió respecto del ensamblado **then**
        (Re)construir $\mathbf{A} = \mathbf{M}/\Delta t - \theta\mathbf{L}$ y $\mathbf{B} = \mathbf{M}/\Delta t + (1-\theta)\mathbf{L}$; marcar el solver para rebuild.
    end if ▷ constantes si Δt es fijo ⇒ se reutilizan (Fase B)
    Construir la fuente vieja $S^n = -I_{\text{ion}}^n + I_{\text{ext}}^n/(\chi C_m)$.
    if `jfnkInitGuessOrder` ≥ 1 y existe paso previo **then**
        $\mathsf{x}_0 \leftarrow \mathsf{V}^n + (\Delta t/\Delta t_{\text{prev}})(\mathsf{V}^n - \mathsf{V}^{n-1})$.
        Recortar x_0 a $[$`jfnkInitGuessVmMin`, `jfnkInitGuessVmMax`$]$.
    else
        $\mathsf{x}_0 \leftarrow \mathsf{V}^n$.
    end if
    Correr el Algoritmo 2, 3 o 4.
    if no convergió y `nonlinearAcceptUnconverged` es false **then**
        Hacer rollback de $V_{\text{m}}, I_{\text{ion}}, \mathbf{s}$ a los snapshots guardados en t^n.
    else
        Committear el iterado no lineal final.
    end if
    Actualizar tiempos de activación y muestras del benchmark.
end while

Algorithm 2 Solve no lineal de Picard.

Entrada: Candidato inicial x_0, snapshot de state $\mathbf{s}^n$.
for $k = 0, \ldots,$ `nonlinearIterations` $- 1$ **do**
    Guardar iterado previo x_{k-1}, $I_{\text{ion},k-1}$, $\mathbf{s}_{k-1}$ para los residuos.
    evaluateNonlinearFields(x_k): ▷ evaluación completa #1 resetear states a $\mathbf{s}^n$; si HO, reconstruir V_{m} en los puntos de cuadratura de Iion (expansión de Taylor con el esténcil LRE); integrar los ODE con `ionicModel->solveODE` en esos puntos; integrar I_{ion} al valor de celda $I_{\text{ion},k}$ por cuadratura de Gauss.
    Construir el RHS $\mathsf{r}(I_{\text{ion},k})$ usando (26).
    Resolver $\mathbf{A}\,\mathsf{V}_* = \mathsf{r}(I_{\text{ion},k})$ con el back-end lineal, reutilizando la factorización si $\mathbf{A}$ no cambió (*Reuse*; Picard tiene $\mathbf{A}$ constante).
    Relajar: $\mathsf{x}_{k+1} \leftarrow \mathsf{x}_k +$ `nonlinearRelaxation`$(\mathsf{V}_* - \mathsf{x}_k)$.
    evaluateNonlinearFields(x_{k+1}) ▷ evaluación completa #2 (para residuos)
    Computar R_{cpl}, $R_{V_{\text{m}}}$, $R_{I_{\text{ion}}}$ y el opcional $R_{\mathbf{s}}$.
    if todos los criterios pedidos pasan y $k + 1 \geq$ `minNonlinearIterations` **then**
        Marcar convergido y salir del bucle.
    end if
end for
Nota: cada iteración hace *dos* evaluaciones completas del campo (reconstrucción LRE + ODE + cuadratura), que dominan el costo por paso; el solve lineal es una fracción menor.

Algorithm 3 Solve no lineal `diagonalIion`.

Entrada: Candidato inicial x_0, snapshot de state $\mathbf{s}^n$.

for $k = 0, \ldots,$ `nonlinearIterations` $- 1$ **do**
  Evaluar states y $I_{\text{ion},k}$ en x_k exactamente como en Picard.
  Aproximar la derivada local $S'_k \approx -[I_{\text{ion}}(\mathsf{x}_k + \varepsilon_d) - I_{\text{ion}}(\mathsf{x}_k)]/\varepsilon_d$, con $\varepsilon_d =$ `diagonalIionEpsilon`.
  Ensamblar $\mathbf{A}_k = \mathbf{A} - \theta\,\mathrm{diag}(S'_k)$ ▷ sólo cambia la diagonal; el patrón se mantiene
  Construir el RHS corregido $\mathsf{r}_k = \mathsf{r}(I_{\text{ion},k}) - \theta\,\mathrm{diag}(S'_k)\mathsf{x}_k$.
  Resolver $\mathbf{A}_k\mathsf{V}_* = \mathsf{r}_k$ refrescando sólo los valores de la matriz (*ValuesOnly*); con `petscReusePreconditioner` se reutiliza además el precondicionador entre iteraciones.
  Relajar, re-evaluar ODEs/I_{ion}, computar residuos y testear convergencia como en Picard.
end for

Algorithm 4 Solve Jacobian-Free Newton–Krylov.

Entrada: Candidato inicial x_0, snapshot de state $\mathbf{s}^n$.

for $k = 0, \ldots,$ `nonlinearIterations` $- 1$ **do**
  Evaluar $\mathsf{F}_k = \mathsf{F}(\mathsf{x}_k)$; esto resetea states a $\mathbf{s}^n$, resuelve los ODEs y recomputa I_{ion}.
  Computar residuos. Si todos los criterios pedidos pasan, marcar convergido y salir.
  Definir el producto matrix-free $\mathcal{J}_k\mathbf{v} = [\mathsf{F}(\mathsf{x}_k + \varepsilon\mathbf{v}) - \mathsf{F}_k]/\varepsilon$, con ε de `jfnkEpsilon` ▷ cada matvec = 1 evaluación de F = 1 integración de ODE.
  if `jfnkPreconditioner` = `diffusion` **then**
    Preparar (una vez / al cambiar Δt) el ILU incompleto de $\mathbf{M} = \mathbf{A}$ como precondicionador por la derecha $\mathbf{M}^{-1}$.
  end if
  Resolver $\mathcal{J}_k\delta_k = -\mathsf{F}_k$ por GMRES reiniciado (back-end `jfnkLinearSolverBackend`: GMRES propio de Eigen o `KSPGMRES`/`MatShell` de PETSc), usando `jfnkMaxKrylovIterations`, `jfnkMaxRestarts`, `jfnkLinearTolerance` y, si corresponde, el precondicionador $\mathbf{M}^{-1}$.
  if `jfnkLineSearch` **then**
    Comenzar con $\alpha =$ `nonlinearRelaxation` y retroceder hasta que Armijo (36) tenga éxito, se agote el presupuesto de intentos, o se alcance `jfnkLineSearchAlphaMin`.
  else
    $\alpha \leftarrow$ `nonlinearRelaxation`.
  end if
  $\mathsf{x}_{k+1} \leftarrow \mathsf{x}_k + \alpha\delta_k$.
  Re-evaluar $\mathsf{F}(\mathsf{x}_{k+1})$, actualizar campos, computar residuos y testear convergencia.
end for

Leyenda. `Azul` = constante Python fijada en `run_convergence.py`; `verde` = clave de diccionario OpenFOAM leída por el solver. Constantes y claves están mapeadas uno-a-uno en la sección 12.

12. Driver manufacturado: `run_convergence.py`

El script de convergencia `/home/pablo/OpenFOAM/pablo-v2312/run/tutorials_electro/highOrderManufacturedF` genera un caso MMS cuyo camino numérico busca coincidir con este solver. Las tablas de abajo describen las constantes expuestas ahí, las entradas de diccionario OpenFOAM que generan, y qué parte de las ecuaciones discretas afectan. Las mismas claves de diccionario las lee `highOrderElectroActivationFoamImplicitPETSc`; solo difieren ítems específicos del problema como campos exactos versus TNNP/estímulo.

12.1. Geometría y barrido de parámetros

Constante Python	Clave OpenFOAM	Ecuación / mecanismo afectado
DIMENSIONS	—	Selecciona el dominio manufacturado 2D o 3D y los stencils LRE correspondientes en P_LEVELS_BY_DIM.
N_VALUES	conteos de celdas de blockMeshDict	Refinamiento espacial. Cambia el error de discretización FV/LRE en el PDE, la cuadratura de Iion y la reconstrucción de states.
DT_VALUES	controlDict.deltaT	Paso de tiempo externo del PDE. También fija el intervalo de integración de cada solve de ODE local.
END_TIME	controlDict.endTime	Tiempo final del test MMS transitorio.
IMPLICIT_SCHEMES	spatialIntegrationProperties.implicitScheme	Elige θ en (13): backwardEuler ($\theta = 1$) o crankNicolson ($\theta = 1/2$).
MASS_MATRIX_VALUES	spatialIntegrationProperties.massMatrix	Elige $\mathbf{M}$ lumped o consistente-LRE en $\mathbf{A} = \mathbf{M}/\Delta t - \theta\mathbf{L}$.
ACTIVE_HO_PAIRS_Vm_States	useHighOrder_Vm / useHighOrder_Iion + LRECoeffs_*	Tupla (nivel-Vm, nivel-Iion/states). El nivel de Vm afecta la difusión y la masa consistente; el nivel de Iion/states afecta dónde se resuelven los ODEs y dónde se integra la fuente no lineal.
P_LEVELS_BY_DIM	highOrderCoeffs	Mapea p1,p2,p3 al orden polinómico LRE N, objetivo de stencil Nn, exponente del peso radial k, y maxStencilSize.

12.2. Método no lineal y controles de convergencia

Constante Python	Clave OpenFOAM	Ecuación / mecanismo afectado
NONLINEAR_METHOD	nonlinearMethod	Selecciona la estrategia no lineal para el acoplamiento PDE/fuente: Picard, JFNK, o diagonalIion.
IMPLICIT_NONLINEAR_ITERATIONS	nonlinearIterations	Número máximo de correcciones no lineales por paso de tiempo del PDE.
IMPLICIT_MIN_NONLINEAR_ITERATIONS	minNonlinearIterations	Número mínimo de correcciones antes de que un test de convergencia pueda parar el bucle.
NONLINEAR_TOLERANCE	nonlinearTolerance	Tolerancia del residuo acoplado del PDE R_{cpl}. Usada directamente por JFNK y diagonalIion; en el solver electro también se reporta para Picard.
NONLINEAR_VM_TOLERANCE	nonlinearVmTolerance	Corrección relativa de la incógnita del PDE V_{m}.

Constante Python	Clave OpenFOAM	Ecuación / mecanismo afectado
NONLINEAR_STATES_TOLERANCE	nonlinearStatesTolerance	Corrección relativa de todos los states del ODE. En el solver MMS estos son u_1, u_2, u_3; en este solver electro son las variables de state del TNNP.
NONLINEAR_REQUIRE_STATES_CONVERGENCE	nonlinearRequireStatesConvergence	Si es true, el residuo de states es parte del criterio de parada.
NONLINEAR_ACCEPT_UNCONVERGED	nonlinearAcceptUnconverged	Si es false, un solve no lineal electro fallido hace rollback de $V_{\mathrm{m}}, I_{\mathrm{ion}}$ y states a t^n.
NONLINEAR_IION_TOLERANCE	nonlinearIionTolerance	Corrección relativa de la fuente no lineal I_{ion}.
NONLINEAR_RELAXATION	nonlinearRelaxation	Factor de mezcla Picard/diagonal o longitud de paso inicial de Newton para JFNK/Armijo.

12.3. Controles de JFNK y de diagonal-Iion

Constante Python	Clave OpenFOAM	Ecuación / mecanismo afectado
JFNK_MAX_KRYLOV_ITERATIONS	jfnkMaxKrylovIterations	Longitud de reinicio m de GMRES(m), controlando la memoria y la calidad de la aproximación de Krylov.
JFNK_MAX_RESTARTS	jfnkMaxRestarts	Número de veces que la base de GMRES puede descartarse y reconstruirse.
JFNK_LINEAR_TOLERANCE	jfnkLinearTolerance	Tolerancia relativa del solve de la corrección de Newton $\mathbf{J}\delta = -\mathsf{F}$.
JFNK_EPSILON	jfnkEpsilon	Perturbación base para los productos matriz-vector matrix-free del Jacobiano. Debe estar por encima del piso de ODE/-ruido pero suficientemente chica para aproximar una derivada.
JFNK_INIT_GUESS_ORDER	jfnkInitGuessOrder	Elige el V_{m} inicial retrasado o linealmente extrapolado para el solve no lineal.
JFNK_INIT_GUESS_VM_MIN/MAX	jfnkInitGuessVmMin/Max	Intervalo de clamp para el valor inicial extrapolado y, si está habilitado, las entradas del ODE.
JFNK_CLAMP_ODE_INPUT	jfnkClampODEInput	Protege el solve del ODE de sondas de diferencias finitas de JFNK no físicas.

Constante Python	Clave OpenFOAM	Ecuación / mecanismo afectado
JFNK_LINE_SEARCH	jfnkLineSearch	Activa/desactiva el retroceso de Armijo.
JFNK_LINE_SEARCH_MAX_ITER	jfnkLineSearchMaxIter	Número máximo de divisiones por la mitad del paso de Newton.
JFNK_LINE_SEARCH_ALPHA_MIN	jfnkLineSearchAlphaMin	Piso para la longitud de paso de Armijo.
JFNK_ARMIJO_C	jfnkArmijoC	Parámetro de descenso suficiente en (36).
DIAGONAL_IION_EPSILON	diagonalIionEpsilon	Paso de diferencias finitas para la derivada diagonal de la fuente en diagonalIion.

12.4. Solver lineal del PDE

Constante Python	Clave OpenFOAM	Ecuación / mecanismo afectado
LINEAR_SOLVER_BACKEND	linearSolverBackend	Elige el back-end ensamblado: Eigen o PETSc (Sección 7).
IMPLICIT_LINEAR_SOLVER	linearSolver	Solver disperso usado por Picard y diagonalIion para $\mathbf{A}\mathsf{V} = \mathsf{r}$ (SparseLU/BiCGSTAB en Eigen; mapeado a KSP/PC en PETSc). JFNK en cambio usa GMRES sobre la envoltura Jacobian-free.
PETSC_LINEAR_KSP_TYPE / PETSC_LINEAR_PC_TYPE	petscLinearKspType / petscLinearPcType	Tipo de KSP y precondicionador cuando el back-end es PETSc (gmres+ilu, cg+gamg, ...).
IMPLICIT_TOLERANCE	tolerance	Tolerancia relativa del solve lineal iterativo.
IMPLICIT_MAX_ITERATIONS	maxIterations	Tope de iteraciones del solve lineal iterativo.

12.5. Integrador del ODE de states

Constante Python	Clave OpenFOAM	Ecuación / mecanismo afectado
STATE_ODE_SOLVER	stateODESolver / timeIntegrationProperties.solver	Integrador de ODE local para states MMS o states TNNP.
STATE_ODE_ABS_TOL	stateODEAbsTol / timeIntegrationProperties.absTol	Tolerancia absoluta del solve del ODE.
STATE_ODE_REL_TOL	stateODERelTol / timeIntegrationProperties.relTol	Tolerancia relativa del ODE. Controla el piso de ruido visto por las diferencias finitas de JFNK.
STATE_ODE_INITIAL_STEP	stateODEInitialStep / initialODEStep	Primer sub-paso RKF45 intentado dentro de un paso de tiempo del PDE.

Constante Python	Clave OpenFOAM	Ecuación / mecanismo afectado
`STATE_ODE_MAX_STEPS`	`stateODEMaxSteps` / `maxSteps`	Máximo de sub-pasos de ODE por solve local.

12.6. Parámetros LRE de alto orden por nivel-p

`P_LEVELS_BY_DIM[dim][p]` da, para cada (dim, p): `N` (orden polinómico), `Nn` (vecinos del stencil), `k` (radio de búsqueda), `maxStencilSize` (tope). El script escribe estos en un bloque anidado `highOrderCoeffs{LRECoeffs_Vm{...}; LRECoeffs_Iion{...}}` cuando HO está habilitado para Vm o I_{ion}.

12.7. Diagnósticos y control de corrida

Constante Python	Clave OpenFOAM	Mecanismo afectado
`WRITE_NONLINEAR_RESIDUALS`	`writeNonlinearResiduals`	Log de residuos on/off.
`KEEP_LOGS_ON_SUCCESS`	—	Switch solo-Python: conservar logs del solver/blockMesh tras corridas exitosas. Los logs de corridas fallidas se conservan siempre.

Para el tutorial de electro-activación el driver análogo también expone parámetros de umbral de activación, parada en P8 y muestreo de la diagonal. No afectan el solve PDE/ODE en sí; afectan solo la parada y el post-procesamiento de los tiempos de activación.

13. Cambios de octubre de 2026 y resultados nuevos

Esta sección reúne lo que cambió en el solver entre agosto y el 2026-10-18, y los tests de la suite de verificación que lo miden (`run/tutorials_electro/electroTestParallelVerification_2D/` y, para el MMS, `mmsTestParallelVerification_2D/`). Las secciones anteriores ya están corregidas donde quedaban falsas; acá está el detalle. Los resultados de la Sección 14 son **anteriores** a todo esto.

13.1. Código común con el solver MMS

Lo que este solver comparte con `highOrderManufacturedFDAImplicitPETScDistributed` se mudó a headers de `src/monodomainCommon/`:

Header	Contenido
`monodomainBase.H`	offset global de fila, validación de opciones, criterio de parada no lineal
`monodomainAssembly.H`	ensamblado de **M** y **K** (ortogonal y de alto orden)
`monodomainPetsc.H`	matrices distribuidas, KSP cacheado, selección de KSP/PC
`monodomainKrylov.H`	GMRES propio y JFNK
`monodomainTimeSchemes.H`	coeficientes θ/BDF, tabla ESDIRK3, driver de V_{m}
`monodomainReconstruction.H`	reconstrucciones a los puntos de Gauss
`openMPControls.H`, `jfnkPreconditionerControls.H`	claves de OpenMP y del precondicionador de JFNK

No son una biblioteca: se incluyen *dentro* del espacio de nombres anónimo de cada `.C`, así que cada ejecutable compila su copia y un cambio ahí cambia los dos solvers. El reporte de código (`codigo_*.pdf`, generado por `mkcodereport.py`) busca cada función en el `.C` y en esos headers.

13.2. BDF1–4 con arranque ESDIRK3

Portados del MMS. Un BDF-k escribe el paso como

$$\Big(\frac{a_0}{\Delta t}\mathbf{M} - \mathbf{L}\Big)\mathsf{V}^{n+1} \;=\; \frac{1}{\Delta t}\mathbf{M}\,\mathsf{H}^{n} + \mathbf{s}^{n+1}, \qquad \mathsf{H}^{n} = -\sum_{j=1}^{k} a_j\,\mathsf{V}^{n+1-j}, \tag{39}$$

es decir, el mismo par de operadores que el esquema θ con $\theta = 1$ y otro escalar delante de $\mathbf{M}$: el ensamblado, la esparsidad y la factorización cacheada no cambian, y la historia entra con un único producto por $\mathbf{M}$. BDF3 y BDF4 son $A(86°)$ y $A(73°)$-estables; BDF5 y BDF6 no se ofrecen (52° y 18°, que el jacobiano de reacción de un frente no respeta).

Los pasos que un BDF-k no puede dar solo — los $k-1$ primeros, los $k-1$ siguientes a un reinicio de la historia y un paso de longitud no nominal (el último recortado, el punto de parada) — se toman con **ESDIRK3(2)4L[2]SA**, de un paso, L-estable y de tercer orden respecto del sistema semi-discreto, que es lo que conserva el orden nominal (`bdfStartup ESDIRK3`, único valor).

La historia se **reinicia** cuando cambia el estímulo (un salto en el forzado que un multipaso no puede cruzar), tras un paso revertido y en un paso de longitud no nominal. El estímulo, además, se **promedia sobre el paso**: la carga inyectada es exacta para cualquier Δt, cuando antes el estímulo duraba un paso de más.

13.3. El driver de V_m de los ODEs

Los states se integran con sub-pasos dentro del paso del PDE, y cada sub-paso necesita V_m en un instante intermedio. Con V_m congelado en el candidato el TNNP daba orden 1 incluso con Crank–Nicolson. Ahora la EDO lo toma de un polinomio de Lagrange en el tiempo (`interpolateVmDriver`):

- `stateODEDriver linear`: la recta entre V^n y el candidato; de segundo orden, topea a todo BDF en 2 (MMS M21: BDF4 2.04).
- `stateODEDriver history` (por defecto): pasa además por los niveles de la historia BDF o los valores de etapa ESDIRK, que el esquema ya guarda; no cuesta evaluaciones extra (M21: BDF4 3.89).

`backwardEuler`, `crankNicolson` y `BDF1` no guardan niveles anteriores a V^n y usan la recta con cualquiera de los dos valores.

13.4. La historia de V_m en los puntos de Gauss

En los caminos de puntos de Gauss (`gaussPointODE` y el frente del híbrido) el driver era siempre la recta, porque subirlo parecía exigir guardar cada nivel en cada punto. Con `stateODEGaussPointHistory true` esos caminos usan el mismo polinomio que el camino por celda *sin guardar nada entre pasos*: cada nivel de la historia (o cada valor de etapa ESDIRK aceptado) se reconstruye en los puntos de Gauss al principio del paso, con el mismo operador que da V^n ahí (`reconstructVmLevelAtIionIntegrationPoints`). Dentro del paso viven $k-1$ arreglos temporales del tamaño de los puntos de Gauss. Sólo actúa con `stateODEDriver history`; el solver lo tiene apagado por defecto y el harness lo enciende.

Sin la clave, BDF3 y BDF4 quedaban en ∼2.4–2.5 en GP (E02); con ella dan su orden (Sección 13.8). El costo lo mide E36, BDF4 a 0.25 mm, $\Delta t = 2{,}5 \times 10^{-5}$ s, 4 ranks, dos repeticiones alternadas:

Modo	lazo sin la clave [s]	lazo con la clave [s]	costo
GP	51.8	56.4	+8.9 %
CC + híbrido	12.6	13.2	+5.1 %

El pico de memoria no cambia de forma medible (173 MB); la memoria temporal teórica es de 24 bytes por punto de Gauss con BDF4, contra 152 de los 19 states del TNNP en ese mismo punto.

13.5. El piso temporal del híbrido, y su arreglo

Con las correcciones anteriores, CC puro y GP convergían en Δt, pero el híbrido quedaba en un **piso** de $\sim 2{,}2 \times 10^{-3}$ en la diferencia de Cauchy que ningún esquema bajaba (todos iguales en el par fino).

Lo que no era. La primera hipótesis fue que las celdas cerca del umbral de `hybridFrontVmRange` entraran y salieran de la banda. El diagnóstico de cambios de modo la descartó: ~ 250 cambios por corrida, constantes al achicar Δt. Una histéresis en la clasificación (`hybridFrontVmRangeExit`) tampoco ayudó, sin limitador las corridas caían en `SIGFPE`, y se eliminó del solver. Agrandar la banda tampoco.

Lo que era. Las celdas suaves rehacían su state de celda en cada paso como el promedio de su reconstrucción en los puntos de Gauss. El promedio de una reconstrucción no es la identidad: para una reconstrucción cuadrática en una celda cuadrada de lado h,

$$\frac{1}{|\Omega_i|}\int_{\Omega_i} \widehat{s}\,\mathrm{d}\mathbf{x} \;=\; s_i + \frac{h^2}{24}\,\Delta s + O(h^4), \tag{40}$$

una difusión de los states aplicada *una vez por paso*, y por lo tanto con coeficiente efectivo $\propto h^2/(24\,\Delta t)$: más fuerte cuanto más chico Δt, que es exactamente un piso en la convergencia temporal.

El arreglo. Las celdas suaves guardan su state de celda, `hybridCellStates`, entre pasos, como CC puro, y sólo el frente vive en los puntos de Gauss; el promedio de Gauss se usa una sola vez, cuando una celda sale de la banda. Sin limitador, CN pasó de $1{,}49 \times 10^{-3}$ a $3{,}72 \times 10^{-4}$ (orden 2.00) y BDF4 de $2{,}80 \times 10^{-4}$ a $5{,}8 \times 10^{-6}$ en el par fino: lo mismo que GP con historia, a la mitad del costo.

Diagnóstico en el log. Al final de cada corrida con híbrido el solver imprime `Hybrid front band: N mode switches; M cells switched more than twice (max K)`. Sin híbrido no imprime nada.

13.6. Otros cambios

- **Criterio de parada común y salida por piso** (Sección 7 y el párrafo de tests de parada): los tres métodos y las etapas ESDIRK usan `nonlinearIterationExit`.
- **Precondicionado por derecha** del KSP ensamblado, para que la tolerancia lineal mida el residuo verdadero.
- **Reconstrucción de K campos a la vez** (`reconstructCellFieldsAtIntegrationPoints`): el stencil MLS se recorre una vez por celda para los 19 states; bit a bit igual a reconstruirlos de a uno.
- **Dirichlet rechazado.** `fixedValue` y `fixedVoltage` se rechazan al arrancar: el operador ensamblaba el término diagonal pero el levantamiento del lado derecho nunca se formaba, así que el borde quedaba fijado a cero. Medido con paredes a -40 mV en bajo orden: 47 mV de error y creciendo, exit 0, sin aviso. Las condiciones aceptadas son `zeroGradient`, `empty` y las de procesador.
- **OpenMP**: las reconstrucciones a los puntos de Gauss tienen hilos, y `Eigen::setNbThreads` ata los productos de Eigen a la misma clave.

13.7. Valores por defecto: solver y harness

Clave	solver	harness de la suite (desde 2026-10-16)
`TNNPNumericalProtection`	`false`	`False`
`stateReconstructLimiter`	`false`	`False`
`stateODEGaussPointHistory`	`false`	`True`
`stateODEDriver`	`history`	`history`
`bdfStartup`	`ESDIRK3`	`ESDIRK3`

13.8. Resultados nuevos

E02: convergencia temporal por esquema y modo de states. 2-D, 0.5 mm, V_m a $t = 10$ ms, Cauchy en $\Delta t = 10^{-4} \ldots 6{,}25 \times 10^{-6}$ s, seis esquemas y cuatro modos, historia en Gauss encendida, sin protección ni limitador (120 corridas). Orden observado con el par más fino:

Esquema	nominal	bajo orden (NO,NO,NO)	CC puro (p3,p3,p3)	CC + híbrido (p3,p3,p3)	GP puro (p3,NO,p3)
`backwardEuler`, `BDF1`	1	0.59	0.82	0.97	0.97
`crankNicolson`	2	1.95	2.00	2.00	2.00
`BDF2`	2	1.75	1.89	1.59	1.59
`BDF3`	3	2.77	2.96	2.93	2.93
`BDF4`	4	4.60	4.60	5.59	5.47
tiempo medio por corrida [s]		148	185	568	1135

`BDF1` da el mismo V_m que `backwardEuler` en todos los casos. El veredicto es FAIL (12/13) por una sola verificación: en CC puro, backwardEuler y BDF1 no achican la diferencia en el par más grueso ($2{,}23 \times 10^{-2}$ y luego $2{,}47 \times 10^{-2}$), saturación de una medida L2 de un frente corrido más que su ancho. Quedan abiertos BDF2 en ∼1.59 en los caminos de Gauss y backwardEuler en 0.59 en bajo orden.

E35: convergencia espacial (Cauchy) del tiempo de activación en P8. Con alto orden T_{P8} tiene que subir al refinar. GP y CC + híbrido lo hacen (33.68, 41.49, 44.11, 44.87 ms en 0.5, 0.25, 0.125 y 0.0625 mm, iguales entre sí, el híbrido 3–6 veces más barato); CC puro no es monótono (47.39, 50.82, 48.61, 45.56 ms): reconstruye los states, casi discontinuos en el frente, y pierde el alto orden ahí. Bajo orden baja (156.7, 66.3, 49.5, 45.8, 45.0 ms). Esta campaña es **anterior** al arreglo del híbrido y a los valores por defecto actuales del harness, y quedó con 8 corridas pendientes en 0.03125 mm.

E36: costo de la historia en Gauss. PASS, ver Sección 13.4.

M02 (MMS, solución exacta): GP con historia. 2-D hexa $N = 40$, p3. Con la clave de la historia en Gauss, el camino GP da, por la vía A (Cauchy) y la B (error contra la solución analítica): CN 2.000/2.000, BDF2 1.979/1.992, BDF3 2.932/2.971, BDF4 3.891/3.908. Sin la clave, por la vía B, BDF4 se estanca en un error de $\sim 2 \times 10^{-10}$. Es la verificación con solución exacta de que la clave lleva el camino de Gauss al orden nominal.

14. Resultados

Nota (2026-10). Las tablas de esta sección se corrieron con el solver de julio–agosto, **anterior** a todos los cambios de la Sección 13 (entre otros: sólo esquema θ, limitador de states encendido por

defecto y el híbrido con el piso temporal de la Sección 13.5). Se conservan como registro y no se volvieron a correr; los resultados con el solver actual están en la Sección 13.8.

Esta sección reúne los resultados actuales para los benchmarks 2D y 3D estilo Niederer. El benchmark 2D (Sección 14.1) se volvió a correr con un barrido `Picard` ampliado que cubre los modos de integración de states y el híbrido consciente del frente (*front-aware*) introducidos en la Sección 5; el benchmark 3D (Sección 14.2) conserva los resultados reportados previamente (`Picard` y `diagonalIion`) y no se modifica aquí. Todas las corridas usan el esquema temporal Crank–Nicolson ($\theta = 1/2$), la matriz de masa consistente de alto orden cuando la LRE está habilitada, y el integrador de ODE de states `RKF45`. El tiempo de reloj de pared (wall-clock) y el RSS pico se definen en la Sección 14.1.1, junto con las magnitudes de costo independientes del hardware.

14.1. Benchmark 2D estilo Niederer

El caso 2D es la reducción planar de 20×8 mm (una sola celda en z, fibras planares). El barrido reportado aquí es exhaustivo en `Picard` y vive bajo `highOrderNiedererEtAl2012ImplicitPETSc/results/example0/2D_run_20260705_110250/`. Cubre el producto de:

- **Malla:** hexaédrica estructurada (`hexa`) y triangular no estructurada (`triangularUnstr`).
- **Híbrido:** `frontAwareHybrid false` (8 configuraciones) y `true` (4 configuraciones).
- **Paso temporal:** $\Delta t \in \{10^{-4}, 10^{-5}, 10^{-6}\}$ s.
- **Malla espacial:** $\Delta x \in \{0{,}5, 0{,}25, 0{,}125, 0{,}1, 0{,}05\}$ mm. Los dos niveles más finos (0,1 y 0,05 mm) se corrieron solo para las configuraciones que alcanzan el régimen convergido; las combinaciones no corridas aparecen como `--` en las tablas.

Cada configuración se identifica por la terna `(Vm, states, Iion)`:

- `Vm`$\in$ {`NO`,`p3`}: orden de la reconstrucción LRE del potencial en la difusión (`NO` = volúmenes finitos clásico).
- `states`$\in$ {`na`,`GP`,`CCp1`,`CCp3`}: modo de integración de states (Sección 5). `na` = no aplica (caso base sin alto orden en I_{ion}); `GP` = `gaussPointODE` (ODE en cada punto de Gauss); `CCp`k = `cellCentredReconstruct` (ODE en el centro de celda + reconstrucción LRE de orden k de los states a los puntos de Gauss).
- `Iion`$\in$ {`NO`,`p1`,`p3`}: orden de la reconstrucción LRE en los puntos de cuadratura iónicos.

Cuando `frontAwareHybrid` está activo, las configuraciones `CCp`k resuelven el ODE en los puntos de Gauss solo en las celdas del frente y por `cellCentredReconstruct` en el resto (Sección 5.2).

14.1.1. Magnitudes de costo computacional

El tiempo de ejecución de una corrida depende de qué otros procesos compartían la máquina, por lo que no es una medida limpia del costo del algoritmo (véase el análisis de contención de hilos en la nota comparativa asociada). Para poder comparar configuraciones de forma reproducible, cada tabla reporta, además del tiempo y la memoria, dos magnitudes *deterministas* (independientes del hardware y de la carga de la máquina):

wall [min] *Tiempo de reloj de pared*: tiempo transcurrido del solver (`setup+loop`) reportado por su temporizador interno, redondeado a minutos. Es lo que "tarda" la corrida, pero **depende de la carga de la máquina** (contención de CPU, oversubscription de hilos).

CPU [min] *Tiempo de CPU* (`User time` de `/usr/bin/time -v`): suma de segundos-CPU consumidos por el proceso en modo usuario a través de todos los hilos. Es menos sensible a la carga que el wall, pero aún se infla cuando los hilos hacen *busy-wait* bajo contención; por eso CPU > wall en las corridas multihilo (p. ej. a $\Delta x = 0{,}125$ mm).

RSS [MiB] *Peak resident set size*: memoria residente máxima del proceso, $\lfloor \text{kB}/1024 \rfloor$. Escala con el tamaño del stencil LRE y con el número de celdas.

N_{nl} *Iteraciones no lineales totales*: suma sobre todos los pasos de tiempo del número de iteraciones de `Picard`. Equivale al número de evaluaciones completas del residuo no lineal — cada una integra los ODEs, evalúa I_{ion} y ensambla/resuelve el PDE lineal. Es **determinista** (dados Δt, la malla y las tolerancias) y por lo tanto la referencia de costo más limpia entre configuraciones.

N_{lin} *Iteraciones lineales totales*: suma de las iteraciones de GMRES a lo largo de todos los solves lineales del PDE. Determinista; mide el trabajo del solver lineal.

Advertencia importante sobre N_{nl} e N_{lin}. Estas dos cuentan iteraciones del *solver*, no el costo de integrar el ODE en cada evaluación. Ese costo por evaluación difiere fuertemente entre modos: `gaussPointODE` integra N_q ODEs rígidos por celda (uno por punto de Gauss), mientras que `cellCentredReconstruct` y las celdas suaves del híbrido integran *uno* por celda. Por eso dos corridas con el mismo N_{nl} pueden diferir varias veces en wall/CPU (véase la discusión del híbrido más abajo): el wall y el CPU capturan el costo de la integración de los ODEs, mientras que N_{nl} e N_{lin} capturan el número de solves. Las cuatro magnitudes son complementarias.

El número de pasos de tiempo n_{steps} (también determinista) no se tabula por separado: para $\Delta t \leq 10^{-5}$ s `Picard` converge típicamente en una iteración por paso, de modo que $N_{\text{nl}} \approx n_{\text{steps}}$; a $\Delta t = 10^{-4}$ s se necesitan varias iteraciones por paso y $N_{\text{nl}} > n_{\text{steps}}$. Las corridas se detienen en cuanto P8 (la esquina lejana) activa, así que $n_{\text{steps}} \approx t_{\text{P8}}/\Delta t$; las corridas marcadas FALLA se abortan en $t = 0{,}02$ s (ignition-timeout).

14.1.2. Malla hexaédrica estructurada, sin híbrido (`frontAwareHybrid false`)

Cuadro 7: Rendimiento por caso — malla hexaédrica estructurada, sin híbrido, todo `Picard`. Para cada terna `(Vm, states, Iion)` las filas listan los Δx [mm] disponibles y las columnas los tres Δt, agrupadas de a tres métricas. FALLA = sin ignición (la corrida se aborta en $t = 0{,}02$ s por el detector de fallo); un valor de P8 implica activación completa del dominio; `--` = combinación no corrida. N_{nl} e N_{lin} son los totales de iteraciones no lineales y lineales (deterministas); wall y CPU en minutos; RSS en MiB.

Δt[s]	10^{-4}	10^{-5}	10^{-6}	10^{-4}	10^{-5}	10^{-6}	10^{-4}	10^{-5}	10^{-6}
`(Vm, states, Iion) = (NO, na, NO)`									
Δx[mm]		P8 [ms]			N_{nl}			N_{lin}	
0.5	153.32	157.15	157.31	6 293	16 136	161 519	18 879	32 272	323 038
0.25	62.99	67.30	67.42	3 510	6 854	68 652	12 748	13 708	137 304
0.125	45.03	50.05	50.26	2 237	5 078	50 971	11 183	15 233	101 942
0.1	42.76	47.86	48.11	2 067	4 851	48 751	11 573	14 553	97 502
0.05	40.22	45.28	45.60	1 618	4 583	46 136	14 560	18 331	138 407
Δx[mm]		wall [min]			CPU [min]			RSS [MiB]	
0.5	13	27	66	13	27	66	181	181	181
0.25	27	46	115	27	46	115	187	188	189
0.125	69	131	339	106	208	1098	213	212	212
0.1	104	201	539	150	302	1546	232	232	232
0.05	325	739	1855	362	842	2835	393	395	388
`(Vm, states, Iion) = (NO, GP, p1)`									
Δx[mm]		P8 [ms]			N_{nl}			N_{lin}	
0.5	91.05	94.34	94.48	4 312	9 722	97 348	12 936	19 444	194 696
0.25	54.86	58.85	58.95	2 829	5 998	60 074	10 126	11 996	120 148
0.125	43.21	48.04	48.27	1 886	4 875	48 965	9 430	14 625	97 930
0.1	41.84	46.79	47.06	1 690	4 743	47 688	9 406	14 229	95 376
0.05	40.14	45.18	45.50	1 611	4 573	46 036	14 497	18 290	138 108
Δx[mm]		wall [min]			CPU [min]			RSS [MiB]	
0.5	18	35	78	18	35	78	181	182	181
0.25	47	85	192	47	85	192	191	193	192
0.125	114	254	603	144	334	1322	228	228	229
0.1	174	406	996	211	503	1975	258	258	257
0.05	656	1535	3652	694	1637	4613	490	492	490

Δt[s]	10^{-4}	10^{-5}	10^{-6}	10^{-4}	10^{-5}	10^{-6}	10^{-4}	10^{-5}	10^{-6}
`(Vm, states, Iion) = (NO, CCp1, p1)`									
Δx[mm]		P8 [ms]			N_{nl}			N_{lin}	
0.5	80.23	FALLA	FALLA	4 079	2 004	20 002	12 237	4 008	40 004
0.25	60.37	FALLA	FALLA	3 428	2 004	20 002	11 872	4 008	40 004
0.125	49.71	FALLA	FALLA	2 470	2 003	20 002	12 350	6 009	40 004
0.1	48.06	FALLA	FALLA	2 284	2 003	20 002	12 225	6 009	40 004
0.05	45.95	FALLA	FALLA	1 845	2 003	20 002	16 603	8 012	60 006
Δx[mm]		wall [min]			CPU [min]			RSS [MiB]	
0.5	11	6	12	11	6	12	183	183	183
0.25	36	23	48	36	23	48	194	196	196
0.125	94	80	185	136	111	485	241	241	241
0.1	148	135	320	199	177	727	277	276	275
0.05	473	530	1230	516	575	1644	567	565	564
`(Vm, states, Iion) = (p3, GP, NO)`									
Δx[mm]		P8 [ms]			N_{nl}			N_{lin}	
0.5	180.55	184.69	184.90	7 027	18 725	187 450	21 081	56 175	562 350
0.25	68.84	73.57	73.73	4 072	7 453	74 677	12 216	22 359	224 031
0.125	48.83	54.36	54.58	2 688	5 503	55 232	8 064	16 437	165 696
Δx[mm]		wall [min]			CPU [min]			RSS [MiB]	
0.5	15	38	169	60	211	2166	192	192	188
0.25	31	50	127	58	109	596	227	228	223
0.125	81	144	398	126	237	1357	373	373	373
`(Vm, states, Iion) = (p3, GP, p1)`									
Δx[mm]		P8 [ms]			N_{nl}			N_{lin}	
0.5	68.92	71.41	71.53	4 056	9 550	95 641	12 168	28 650	286 923
0.25	58.81	63.12	63.27	3 120	6 391	64 047	9 360	19 173	192 141
0.125	46.33	51.39	51.61	2 216	5 204	52 246	6 648	15 599	156 738
Δx[mm]		wall [min]			CPU [min]			RSS [MiB]	
0.5	20	38	138	69	155	1295	193	193	193
0.25	52	96	260	91	174	1038	231	231	231
0.125	132	273	689	170	366	1581	385	385	383
`(Vm, states, Iion) = (p3, GP, p3)`									
Δx[mm]		P8 [ms]			N_{nl}			N_{lin}	
0.5	33.09	36.31	36.56	1 129	3 717	37 404	3 387	11 151	112 212
0.25	40.22	44.20	44.43	1 369	4 494	45 160	4 107	13 482	135 480
0.125	40.84	45.40	45.66	1 392	4 604	46 293	4 176	13 812	138 879
Δx[mm]		wall [min]			CPU [min]			RSS [MiB]	
0.5	21	56	141	35	101	595	198	199	199
0.25	96	255	622	113	310	1171	252	252	252
0.125	335	932	2228	359	1012	3026	467	461	463
`(Vm, states, Iion) = (p3, CCp1, p1)`									
Δx[mm]		P8 [ms]			N_{nl}			N_{lin}	
0.5	91.40	FALLA	FALLA	4 636	2 004	20 002	13 908	6 012	60 006
0.25	66.46	FALLA	FALLA	4 168	2 004	20 002	12 504	6 012	60 006
0.125	52.98	FALLA	FALLA	2 925	2 003	20 002	8 775	6 009	60 006
0.1	50.47	FALLA	FALLA	2 554	2 003	20 002	10 214	6 009	60 006
0.05	46.51	FALLA	FALLA	1 879	2 003	20 002	9 393	6 009	60 006
Δx[mm]		wall [min]			CPU [min]			RSS [MiB]	
0.5	15	7	24	71	31	267	196	195	194
0.25	46	24	62	96	48	304	237	236	238
0.125	111	82	201	161	116	544	413	413	413
0.1	160	133	322	217	175	739	538	534	533
0.05	470	523	1270	513	569	1717	1433	1416	1416
`(Vm, states, Iion) = (p3, CCp3, p3)`									
Δx[mm]		P8 [ms]			N_{nl}			N_{lin}	
0.5	66.63	FALLA	FALLA	2 788	2 004	20 002	8 364	6 012	60 006
0.25	60.80	FALLA	FALLA	2 942	2 004	20 002	8 826	6 012	60 006
0.125	52.20	FALLA	FALLA	2 455	2 003	20 002	7 365	6 009	60 006
0.1	50.15	FALLA	–	2 231	2 003	–	8 922	6 009	–
0.05	46.48	–	–	1 863	–	–	9 313	–	–
Δx[mm]		wall [min]			CPU [min]			RSS [MiB]	

Δt[s]	10^{-4}	10^{-5}	10^{-6}	10^{-4}	10^{-5}	10^{-6}	10^{-4}	10^{-5}	10^{-6}
0.5	10	8	33	44	32	273	203	203	203
0.25	37	28	101	73	52	344	271	270	271
0.125	107	97	374	149	131	752	541	538	538
0.1	163	161	–	211	202	–	743	741	–
0.05	540	–	–	581	–	–	2416	–	–

14.1.3. Malla hexaédrica estructurada, híbrido front-aware (`frontAwareHybrid true`)

Cuadro 8: Rendimiento por caso — malla hexaédrica estructurada, híbrido front-aware, todo `Picard`. Para cada terna `(Vm, states, Iion)` las filas listan los Δx [mm] disponibles y las columnas los tres Δt, agrupadas de a tres métricas. FALLA = sin ignición (la corrida se aborta en $t = 0{,}02$ s por el detector de fallo); un valor de P8 implica activación completa del dominio; `--` = combinación no corrida. N_{nl} e N_{lin} son los totales de iteraciones no lineales y lineales (deterministas); wall y CPU en minutos; RSS en MiB.

Δt[s]	10^{-4}	10^{-5}	10^{-6}	10^{-4}	10^{-5}	10^{-6}	10^{-4}	10^{-5}	10^{-6}
`(Vm, states, Iion) = (NO, na, NO)`									
Δx[mm]		P8 [ms]			N_{nl}			N_{lin}	
0.5	153.32	157.15	157.31	6 293	16 136	161 519	18 879	32 272	323 038
0.25	62.99	67.30	67.42	3 510	6 854	68 652	12 748	13 708	137 304
0.125	45.03	50.05	50.26	2 237	5 078	50 971	11 183	15 233	101 942
Δx[mm]		wall [min]			CPU [min]			RSS [MiB]	
0.5	13	27	69	13	27	69	181	181	181
0.25	28	47	117	28	47	117	188	189	188
0.125	69	131	341	108	211	1119	212	212	212
`(Vm, states, Iion) = (NO, CCp1, p1)`									
Δx[mm]		P8 [ms]			N_{nl}			N_{lin}	
0.5	91.05	94.34	94.48	4 320	9 722	97 350	12 960	19 444	194 699
0.25	54.86	58.85	58.95	2 826	5 998	60 076	10 117	11 996	120 152
0.125	43.20	48.05	48.29	1 904	4 876	48 988	9 519	14 627	97 976
0.1	41.85	46.79	47.07	1 695	4 744	47 702	9 430	14 232	95 403
0.05	40.14	45.18	45.51	1 611	4 573	46 040	14 497	18 291	138 119
Δx[mm]		wall [min]			CPU [min]			RSS [MiB]	
0.5	12	23	57	12	23	57	182	183	183
0.25	27	50	126	27	50	126	195	196	195
0.125	63	136	385	96	213	1136	240	241	240
0.1	94	220	630	132	319	1609	278	279	277
0.05	340	825	2208	379	927	3175	564	568	570
`(Vm, states, Iion) = (p3, CCp1, p1)`									
Δx[mm]		P8 [ms]			N_{nl}			N_{lin}	
0.5	68.93	71.44	71.56	4 041	9 554	95 681	12 123	28 662	287 043
0.25	58.82	63.11	63.27	3 120	6 390	64 044	9 360	19 170	192 132
0.125	46.33	51.39	51.61	2 218	5 204	52 250	6 654	15 598	156 750
Δx[mm]		wall [min]			CPU [min]			RSS [MiB]	
0.5	13	27	114	62	143	1277	195	195	195
0.25	31	56	175	69	134	955	238	238	237
0.125	72	149	452	111	240	1371	414	414	414
`(Vm, states, Iion) = (p3, CCp3, p3)`									
Δx[mm]		P8 [ms]			N_{nl}			N_{lin}	
0.5	33.09	36.30	36.51	1 130	3 715	37 350	3 390	11 145	112 050
0.25	40.22	44.22	44.54	1 371	4 496	45 266	4 113	13 488	135 798
0.125	40.84	45.43	45.90	1 391	4 607	46 527	4 173	13 821	139 581
0.1	40.72	45.48	45.98	1 402	4 609	46 573	5 607	13 827	139 719
0.05	40.05	45.04	45.52	1 604	4 558	46 051	8 019	13 674	138 153
Δx[mm]		wall [min]			CPU [min]			RSS [MiB]	
0.5	6	17	64	20	62	519	202	202	202

Δt[s]	10^{-4}	10^{-5}	10^{-6}	10^{-4}	10^{-5}	10^{-6}	10^{-4}	10^{-5}	10^{-6}
0.25	20	60	216	37	115	767	267	267	267
0.125	64	205	845	89	298	1841	530	530	530
0.1	104	306	1178	135	403	2147	726	725	726
0.05	419	1123	4211	456	1224	5239	2370	2367	2351

14.1.4. Malla triangular no estructurada, sin híbrido

Cuadro 9: Rendimiento por caso — malla triangular no estructurada, sin híbrido, todo `Picard`. Para cada terna `(Vm, states, Iion)` las filas listan los Δx [mm] disponibles y las columnas los tres Δt, agrupadas de a tres métricas. FALLA = sin ignición (la corrida se aborta en $t = 0{,}02$ s por el detector de fallo); un valor de P8 implica activación completa del dominio; `--` = combinación no corrida. N_{nl} e N_{lin} son los totales de iteraciones no lineales y lineales (deterministas); wall y CPU en minutos; RSS en MiB.

Δt[s]	10^{-4}	10^{-5}	10^{-6}	10^{-4}	10^{-5}	10^{-6}	10^{-4}	10^{-5}	10^{-6}
`(Vm, states, Iion) = (NO, na, NO)`									
Δx[mm]		P8 [ms]			N_{nl}			N_{lin}	
0.5	55.81	59.70	59.88	3 000	6 060	60 760	12 000	18 179	121 520
0.25	45.18	50.28	50.51	2 231	5 095	51 168	15 615	16 180	102 336
0.125	40.93	46.00	46.30	1 662	4 657	46 858	18 281	23 284	140 574
Δx[mm]		wall [min]			CPU [min]			RSS [MiB]	
0.5	15	25	61	15	25	61	183	183	183
0.25	45	88	222	72	150	848	197	197	198
0.125	128	304	808	166	407	1803	246	247	247
`(Vm, states, Iion) = (NO, GP, p1)`									
Δx[mm]		P8 [ms]			N_{nl}			N_{lin}	
0.5	55.81	59.70	59.88	3 000	6 060	60 760	12 000	18 179	121 520
0.25	45.18	50.28	50.51	2 231	5 095	51 168	15 615	16 180	102 336
0.125	40.93	46.00	46.30	1 662	4 657	46 858	18 281	23 284	140 574
Δx[mm]		wall [min]			CPU [min]			RSS [MiB]	
0.5	15	25	60	15	25	60	183	184	183
0.25	45	88	231	73	150	853	197	197	198
0.125	118	297	687	147	389	1482	249	248	250
`(Vm, states, Iion) = (NO, CCp1, p1)`									
Δx[mm]		P8 [ms]			N_{nl}			N_{lin}	
0.5	55.81	59.70	59.88	3 000	6 060	60 760	12 000	18 179	121 520
0.25	45.18	50.28	50.51	2 231	5 095	51 168	15 615	16 180	102 336
0.125	40.93	46.00	46.30	1 662	4 657	46 858	18 281	23 284	140 574
Δx[mm]		wall [min]			CPU [min]			RSS [MiB]	
0.5	14	25	64	14	25	64	185	185	185
0.25	43	86	243	71	149	867	203	203	204
0.125	131	320	874	169	421	1891	272	271	273
`(Vm, states, Iion) = (p3, GP, NO)`									
Δx[mm]		P8 [ms]			N_{nl}			N_{lin}	
0.5	75.42	57.19	57.29	4 320	7 956	79 678	12 960	23 868	239 034
0.25	51.56	56.76	56.99	2 981	5 741	57 626	10 672	17 223	172 878
0.125	43.26	48.42	48.66	2 144	4 904	49 263	10 719	14 712	147 789
Δx[mm]		wall [min]			CPU [min]			RSS [MiB]	
0.5	20	31	88	50	81	608	203	202	200
0.25	51	85	217	71	122	584	272	272	268
0.125	150	304	770	189	398	1645	561	560	562
`(Vm, states, Iion) = (p3, GP, p1)`									
Δx[mm]		P8 [ms]			N_{nl}			N_{lin}	
0.5	75.42	57.19	57.29	4 320	7 956	79 678	12 960	23 868	239 034
0.25	51.56	56.76	56.99	2 981	5 741	57 626	10 672	17 223	172 878
0.125	43.26	48.42	48.66	2 144	4 904	49 263	10 719	14 712	147 789

Δt[s]	10^{-4}	10^{-5}	10^{-6}	10^{-4}	10^{-5}	10^{-6}	10^{-4}	10^{-5}	10^{-6}
Δx[mm]		wall [min]			CPU [min]			RSS [MiB]	
0.5	23	37	124	75	134	1090	204	204	204
0.25	57	96	269	93	166	971	273	273	273
0.125	151	299	804	189	385	1678	563	561	563
`(Vm, states, Iion) = (p3, GP, p3)`									
Δx[mm]		P8 [ms]			N_{nl}			N_{lin}	
0.5	38.88	42.26	42.47	1 930	4 291	43 113	5 790	12 873	129 339
0.25	38.93	43.26	43.52	1 546	4 388	44 119	5 332	13 164	132 357
0.125	39.80	44.66	44.96	1 569	4 526	45 549	7 844	13 578	136 647
Δx[mm]		wall [min]			CPU [min]			RSS [MiB]	
0.5	40	75	197	64	128	723	211	211	211
0.25	127	296	702	146	350	1240	300	297	300
0.125	456	1069	2618	486	1149	3438	656	665	657
`(Vm, states, Iion) = (p3, CCp1, p1)`									
Δx[mm]		P8 [ms]			N_{nl}			N_{lin}	
0.5	75.42	57.19	57.29	4 320	7 956	79 678	12 960	23 868	239 034
0.25	51.56	56.76	56.99	2 981	5 741	57 626	10 672	17 223	172 878
0.125	43.26	48.42	48.66	2 144	4 904	49 263	10 719	14 712	147 789
Δx[mm]		wall [min]			CPU [min]			RSS [MiB]	
0.5	24	38	135	77	136	1103	205	205	205
0.25	59	101	298	95	171	1000	290	290	291
0.125	173	367	988	221	472	2089	616	619	619
`(Vm, states, Iion) = (p3, CCp3, p3)`									
Δx[mm]		P8 [ms]			N_{nl}			N_{lin}	
0.5	61.74	FALLA	FALLA	2 814	2 004	20 002	8 442	6 012	60 006
0.25	54.04	FALLA	FALLA	2 486	2 003	20 002	7 708	6 009	60 006
0.125	48.77	FALLA	FALLA	1 972	2 003	20 002	9 858	6 009	60 006
Δx[mm]		wall [min]			CPU [min]			RSS [MiB]	
0.5	20	15	52	54	40	296	220	219	219
0.25	67	59	188	97	84	433	334	334	334
0.125	208	230	681	253	275	1050	791	791	791

14.1.5. Malla triangular no estructurada, híbrido front-aware

Cuadro 10: Rendimiento por caso — malla triangular no estructurada, híbrido front-aware, todo `Picard`. Para cada terna `(Vm, states, Iion)` las filas listan los Δx [mm] disponibles y las columnas los tres Δt, agrupadas de a tres métricas. FALLA = sin ignición (la corrida se aborta en $t = 0{,}02\,\text{s}$ por el detector de fallo); un valor de P8 implica activación completa del dominio; `--` = combinación no corrida. N_{nl} e N_{lin} son los totales de iteraciones no lineales y lineales (deterministas); wall y CPU en minutos; RSS en MiB.

Δt[s]	10^{-4}	10^{-5}	10^{-6}	10^{-4}	10^{-5}	10^{-6}	10^{-4}	10^{-5}	10^{-6}
`(Vm, states, Iion) = (NO, na, NO)`									
Δx[mm]		P8 [ms]			N_{nl}			N_{lin}	
0.5	55.81	59.70	59.88	3 000	6 060	60 760	12 000	18 179	121 520
0.25	45.18	50.28	50.51	2 231	5 095	51 168	15 615	16 180	102 336
0.125	40.93	46.00	46.30	1 662	4 657	46 858	18 281	23 284	140 574
Δx[mm]		wall [min]			CPU [min]			RSS [MiB]	
0.5	14	26	61	14	26	61	183	184	183
0.25	45	89	231	72	151	855	197	197	198
0.125	135	321	782	171	421	1797	247	247	247
`(Vm, states, Iion) = (NO, CCp1, p1)`									
Δx[mm]		P8 [ms]			N_{nl}			N_{lin}	
0.5	55.81	59.70	59.88	3 000	6 060	60 760	12 000	18 179	121 520
0.25	45.18	50.28	50.51	2 231	5 095	51 168	15 615	16 180	102 336
0.125	40.93	46.00	46.30	1 662	4 657	46 858	18 281	23 284	140 574

Δt[s]	10^{-4}	10^{-5}	10^{-6}	10^{-4}	10^{-5}	10^{-6}	10^{-4}	10^{-5}	10^{-6}
Δx[mm]	wall [min]			CPU [min]			RSS [MiB]		
0.5	15	26	66	15	26	66	185	186	185
0.25	46	90	241	73	152	864	204	204	206
0.125	138	330	878	174	429	1880	273	273	272
`(Vm, states, Iion) = (p3, CCp1, p1)`									
Δx[mm]	P8 [ms]			N_{nl}			N_{lin}		
0.5	75.42	57.19	57.29	4 320	7 956	79 678	12 960	23 868	239 034
0.25	51.56	56.76	56.99	2 981	5 741	57 626	10 672	17 223	172 878
0.125	43.26	48.42	48.66	2 144	4 904	49 263	10 719	14 712	147 789
Δx[mm]	wall [min]			CPU [min]			RSS [MiB]		
0.5	23	37	132	75	134	1097	205	205	205
0.25	57	98	299	93	168	999	289	289	291
0.125	162	314	978	205	413	2070	618	619	620
`(Vm, states, Iion) = (p3, CCp3, p3)`									
Δx[mm]	P8 [ms]			N_{nl}			N_{lin}		
0.5	38.88	42.27	42.55	1 930	4 293	43 188	5 790	12 879	129 564
0.25	38.93	43.30	43.77	1 543	4 391	44 368	5 321	13 173	133 104
0.125	39.80	44.71	45.30	1 570	4 531	45 883	7 850	13 593	137 649
0.1	39.86	44.81	45.40	1 575	4 535	45 925	9 449	13 605	137 775
0.05	39.62	44.61	–	1 586	4 513	–	15 856	18 052	–
Δx[mm]	wall [min]			CPU [min]			RSS [MiB]		
0.5	15	29	108	38	81	631	219	219	219
0.25	41	103	355	59	156	897	331	331	331
0.125	154	388	1409	189	489	2376	781	793	793
0.1	236	578	2135	271	675	3122	1126	1128	1126
0.05	896	2216	–	955	2341	–	3925	3883	–

Discusión de los resultados 2D (`Picard`). Todo el barrido usa `Picard`; los resultados de `diagonalIion` y `JFNK` en 2D quedan pendientes. Respecto de la versión anterior de este reporte, el barrido se extendió a $\Delta x \in \{0{,}1, 0{,}05\}$ mm en las configuraciones que llegan al régimen convergido, lo que permite por primera vez fijar un valor de referencia y medir el error de cada configuración contra él.

Valor convergido de referencia. Sobre malla hexaédrica, las cuatro rutas que alcanzan $\Delta x = 0{,}05$ mm con $\Delta t = 10^{-6}$ s coinciden en

$$t_{\text{P8}}^{\star} \simeq 45{,}5\,\text{ms}, \tag{41}$$

con una dispersión de solo 0,10 ms (0,2 %): 45,60 ms (`(NO,na,NO)`), 45,50 ms (`(NO,GP,p1)`), 45,51 ms (`(NO,CCp1,p1)` híbrido) y 45,52 ms (`(p3,CCp3,p3)` híbrido). Que cuatro discretizaciones con órdenes espaciales, tratamientos de la fuente iónica y modos de integración de states distintos converjan al mismo número es la verificación más fuerte del barrido: el solver es consistente y las diferencias entre configuraciones a mallas gruesas son error de discretización, no sesgos del modelo. Todos los porcentajes de error que siguen se miden contra (41).

1. **El alto orden en la fuente iónica acelera y afila el frente.** Subir `Iion` de `NO` a `p3` baja monótonamente el tiempo de activación: en hexa a $\Delta t = 10^{-6}$, $\Delta x = 0{,}25$ mm, P8 pasa de 67,4 ms (`(NO,na,NO)`) a 44,4 ms (`(p3,GP,p3)`). Es el mismo efecto de alto-orden-en-la-fuente de los reportes explícito e implícito previos, ahora resuelto con `gaussPointODE`.

2. **Las dos familias convergen desde lados opuestos, y a velocidades muy distintas.** Con $\Delta t = 10^{-5}$ s fijo y refinando Δx en hexa:

Δx [mm]	0.5	0.25	0.125	0.1	0.05
`(NO,na,NO)`	157.15	67.30	50.05	47.86	45.28
`(NO,GP,p1)`	94.34	58.85	48.04	46.79	45.18
`(p3,CCp3,p3)` híbrido	36.30	44.22	45.43	45.48	45.04

Las rutas de fuente de bajo orden se acercan a (41) *por arriba* y muy despacio: `(NO,na,NO)` todavía está 10 % alto a 0,125 mm y necesita 0,05 mm para bajar del 1 %. La ruta de alto orden en la fuente se acerca *por abajo* y ya está dentro del 0,2 % a 0,125 mm. En términos de celdas: la ruta de alto orden alcanza la precisión objetivo con 16× menos celdas.

3. $\Delta t = 10^{-5}$ **s ya es suficiente;** 10^{-6} **s es dinero tirado.** Sobre todas las corridas que activan, $|P8(10^{-6}) - P8(10^{-5})| \leq 0{,}5$ ms ($\lesssim 1$ %), mientras que el costo se multiplica por entre 4 y 10: en hexa `(p3,CCp3,p3)` híbrido a 0,125 mm el wall pasa de 205 a 845 min y N_{nl} de 4 607 a 46 527, para mover P8 de 45,43 a 45,90 ms — es decir, *alejándose* del valor convergido. En cambio $\Delta t = 10^{-4}$ s **no** es suficiente: subestima P8 en $\sim 4{,}6$ ms (10 %) en la ruta de alto orden.

4. **Cuidado con la cancelación de errores a** Δt **grueso.** La combinación `(NO,na,NO)`, $\Delta x = 0{,}125$ mm, $\Delta t = 10^{-4}$ s da 45,03 ms, aparentemente un error de 1 %. Es una coincidencia: al refinar Δt a la misma malla el resultado se va a 50,05 y 50,26 ms (10 %). El error temporal (que subestima P8) cancela al error espacial (que lo sobreestima). **Un punto de operación solo es válido si es estable en** Δt; verificar la proximidad al valor convergido no alcanza.

5. `cellCentredReconstruct` **puro falla al refinar** Δt**.** En hexa, las configuraciones `CCp1`/`CCp3` sin híbrido activan a $\Delta t = 10^{-4}$ s pero **no encienden** (FALLA, $N_a = 0$, aborto por ignition-timeout en $t = 0{,}02$ s) a $\Delta t = 10^{-5}$ y 10^{-6} s, y ahora se confirma que el fallo persiste también a $\Delta x = 0{,}1$ y 0,05 mm: **no es un problema de resolución espacial**. En triangular ocurre lo mismo con `(p3,CCp3,p3)`. El comportamiento está gobernado enteramente por Δt y es independiente de Δx sobre un rango de 10×; su causa está bajo análisis y no se atribuye aquí.

6. **El híbrido front-aware rescata a** `CC` **a costo de** `GP` **parcial.** Con `frontAwareHybrid true`, las mismas configuraciones `CCp`k activan a *todos* los Δt, con tiempos de activación prácticamente iguales a los de `gaussPointODE` pero mucho más baratos. A $\Delta x = 0{,}125$ mm, $\Delta t = 10^{-5}$ s: `(p3,CCp3,p3)` híbrido da 45,43 ms en 205 min, contra 45,40 ms en 932 min de `(p3,GP,p3)` sin híbrido — misma respuesta, 4,5× más barato, con el mismo N_{nl} ($\approx 4\,600$). La reducción viene exactamente de lo advertido arriba: el híbrido integra los N_q ODEs por celda solo en la banda del frente y uno por celda en el resto, así que a igual número de iteraciones no lineales el trabajo de ODE por evaluación es mucho menor. El híbrido combina, entonces, la *corrección* de `GP` con buena parte del *ahorro* de `CC`.

7. **Escalado del costo.** N_{nl} y N_{lin} crecen ∼linealmente al refinar Δt (más pasos). El wall crece aproximadamente como Δx^{-2} (número de celdas en 2D): en `(p3,CCp3,p3)` híbrido a $\Delta t = 10^{-5}$, pasar de 0,125 a 0,05 mm (6,25× celdas) lleva el wall de 205 a 1 123 min (5,5×) y la RSS de 530 a 2 367 MiB (4,5×). El caso más caro del barrido es `(p3,GP,p3)` sin híbrido en hexa a $\Delta t = 10^{-6}$, $\Delta x = 0{,}125$ mm: 2 228 min (~ 37 h). La RSS crece con el stencil LRE y con $1/\Delta x^2$, y es esencialmente independiente de Δt.

8. **Wall vs. CPU: por qué se reportan las magnitudes deterministas.** A $\Delta x \leq 0{,}125$ mm el CPU excede al wall (ejecución multihilo bajo contención; p. ej. `(NO,na,NO)` hexa $\Delta t = 10^{-6}$: wall 339 min, CPU 1098 min), mientras que a mallas gruesas coinciden. Esa divergencia es precisamente el ruido dependiente de la máquina que motiva usar N_{nl} e N_{lin} como referencia de costo reproducible.

9. **Malla triangular.** El patrón cualitativo se mantiene (fallo de `CC` puro, rescate por el híbrido, alto orden en I_{ion} que acelera). Tres observaciones propias: (i) en triangular, las configuraciones con `Iion=p1` coinciden *exactamente* con el baseline `(NO,na,NO)` (mismo P8, mismo N_{nl}, mismo N_{lin}) — la reconstrucción $p1$ de I_{ion} no aporta sobre esta malla no estructurada, y solo `p3` separa; (ii) el valor convergido triangular, $\simeq 44{,}6$ ms para `(p3,CCp3,p3)` híbrido a 0,05 mm, queda ~ 2 % por debajo del hexaédrico (41) — una diferencia de malla genuina, no de método; (iii) el mayor número de celdas por Δx (5 954 vs 2 560 a 0,25 mm) encarece proporcionalmente todas las magnitudes, de modo que a igual Δx el caso triangular cuesta entre 1,9 y 2,4× más wall.

Configuración recomendada. Combinando exactitud y costo, y tomando (41) como referencia, el barrido identifica un punto de operación claramente dominante y dos alternativas según el presupuesto:

Uso	Terna / híbrido	Δx [mm]	Δt [s]	P8 [ms]	error	wall [min]
Recomendada	`(p3,CCp3,p3)` + híbrido	0.125	10^{-5}	45.43	0,2 %	**205**
Exploración rápida	`(p3,CCp3,p3)` + híbrido	0.25	10^{-5}	44.22	2,8 %	**60**
Referencia	`(p3,CCp3,p3)` + híbrido	0.05	10^{-6}	45.52	0,05 %	4 211
Alternativas al mismo nivel de error, para comparación:						
Bajo orden	`(NO,na,NO)`	0.05	10^{-5}	45.28	0,5 %	739
Alto orden sin híbrido	`(p3,GP,p3)`	0.125	10^{-5}	45.40	0,2 %	932

Por qué esa combinación. Los tres ingredientes son necesarios y cada uno resuelve un problema distinto:

- `Iion=p3` con states `CCp3` da la convergencia espacial rápida (punto 2): es lo que permite parar en 0,125 mm en vez de 0,05 mm.
- `frontAwareHybrid true` es lo que hace que `CCp3` *encienda* (punto 5) y, a la vez, evita pagar los N_q ODEs por celda en todo el dominio (punto 6).
- $\Delta t = 10^{-5}$ s es el mínimo que da estabilidad temporal sin gastar de más (punto 3).

Ganancia. Contra la referencia totalmente resuelta, la configuración recomendada es 20× más barata (205 frente a 4 211 min) por 0,2 % de error. Contra la alternativa de bajo orden a igual exactitud es 3,6× más barata (205 frente a 739 min) y usa 16× menos celdas; contra la misma física sin híbrido, 4,5× más barata. Si solo se necesita un barrido cualitativo, bajar a 0,25 mm cuesta 60 min (70× menos que la referencia) a cambio de un error del 2,8 %, que sigue siendo menor que el 10 % del baseline clásico a 0,125 mm.

Qué evitar. (i) `CCp`k sin híbrido a cualquier $\Delta t \leq 10^{-5}$ s: no enciende, y el fallo no se arregla refinando la malla. (ii) $\Delta t = 10^{-4}$ s como punto de producción: puede dar un número aparentemente bueno por cancelación de errores (punto 4) sin ser reproducible al refinar. (iii) $\Delta t = 10^{-6}$ s: multiplica el costo por 4–10 sin mejorar la respuesta. (iv) `(p3,GP,p3)` sin híbrido: da la misma respuesta que la recomendada pero 4,5× más caro.

14.2. Benchmark 3D estilo Niederer

El barrido 3D sigue la geometría de losa publicada de Niederer ($20 \times 3 \times 7$ mm, $40 \times 6 \times 14$ celdas a $\Delta x = 0{,}5$ mm). Al momento de escribir, solo los barridos `diagonalIion` y `Picard` incluyen corridas 3D, todas a $\Delta x = 0{,}5$ mm. El barrido `JFNK` 3D aún no se ha corrido.

Cuadro 11: Tiempo de activación P8 3D [ms] a $\Delta x = 0{,}5$ mm, $\Delta t = 10^{-4}$ s. Las columnas son los métodos no lineales disponibles.

Vm	Iion	`Picard`	`diagonalIion`
NO	NO	136.67	136.61
NO	p1	33.50	33.39
NO	p3	30.59	30.51
p3	NO	149.13	149.06
p3	p1	32.44	32.36
p3	p3	29.63	NC

Cuadro 12: Tiempo de reloj de pared del solver 3D [min] a $\Delta x = 0{,}5$ mm, $\Delta t = 10^{-4}$ s.

Vm	Iion	`Picard`	`diagonalIion`
NO	NO	80	82
NO	p1	168	229
NO	p3	1489	2159
p3	NO	163	143
p3	p1	167	269
p3	p3	1487	NC

Cuadro 13: Tamaño de conjunto residente pico 3D [MiB] a $\Delta x = 0{,}5$ mm, $\Delta t = 10^{-4}$ s.

Vm	Iion	`Picard`	`diagonalIion`
NO	NO	156	157
NO	p1	211	211
NO	p3	787	849
p3	NO	1123	1121
p3	p1	1164	1162
p3	p3	1565	NC

Discusión de los resultados 3D. En la losa 3D gruesa ($\Delta x = 0{,}5$ mm), activar la LRE solo en la fuente iónica cambia el tiempo de activación de $\approx 136{,}7$ ms (`NO/NO`) a $\approx 33{,}5$ ms (`NO/p1`) y $\approx 30{,}6$ ms (`NO/p3`) — el mismo efecto cualitativo de alto-orden-en-la-fuente ya visto en el barrido 2D y en los reportes Niederer explícito/implícito. El caso totalmente de alto orden (`p3/p3`) alcanza 29,63 ms con Picard (la contraparte diagonalIion aún no se ha completado). Los dos métodos no lineales nuevamente coinciden mejor que 0,1 ms para cada entrada donde ambos están disponibles; Picard es generalmente más rápido, con la brecha más notable en el caso más pesado (`NO/p3`: 1489 min para Picard vs. 2159 min para diagonalIion). El RSS pico escala con el tamaño del stencil LRE y es esencialmente independiente del método. El conjunto de datos es demasiado ralo en Δx para sacar conclusiones cuantitativas sobre convergencia de malla en 3D; la brecha inmediata a llenar es la entrada faltante `p3/p3` de diagonalIion y un barrido JFNK 3D a la misma malla.

A. Tabla de dimensiones

Campo	`dimensionSet` de OpenFOAM	SI
`Vm`	$[1, 2, -3, 0, 0, -1, 0]$	volt
`Iion`	`dimVoltage/dimTime`	$\mathrm{V\,s^{-1}}$
`externalStimulusCurrent`	`dimCurrent/dimVolume`	$\mathrm{A\,m^{-3}}$
`conductivity`	`dimCurrent^2 dimTime^3 / (dimMass dimVolume)`	$\mathrm{S\,m^{-1}}$
`chi`	`dimless/dimLength`	$\mathrm{m^{-1}}$
`Cm`	`dimCurrent dimTime / (dimVoltage dimArea)`	$\mathrm{F\,m^{-2}}$

B. Mapa de archivos

Ruta (relativa a `cardiacFoam-pc/`)	Rol
`applications/solvers/highOrderElectroActivationFoamImplicitPETScDistributed/`	Unidad de traducción principal (bucle temporal, solvers no lineales, back-ends lineales Eigen/PETSc, operadores distribuidos `MPIAIJ`, reconstrucción LRE, seguimiento de activación).
`applications/solvers/highOrderElectroActivationFoamImplicitPETSc/`	Antecesor serie, y `...PETScParallel` el paso intermedio (stencils paralelos, álgebra serie). Se conservan como referencias (Sección D).
`applications/solvers/highOrderManufacturedFDAImplicitPETScDistributed/`	Herramienta de solución manufacturada que comparte el diseño paralelo; lleva el intercambio de estabilización en caras acopladas y los diagnósticos `CF_DUMP_K` / `CF_DUMP_STENCILS` (Sección D.4).
`src/highOrderAdapter/highOrderInterp.H`	Adaptador sobre el `movingLeastSquares` de solids4foam: stencils con columnas globales, intercambio de halo, coeficientes de reconstrucción.
`src/ionicModels/`	Modelos iónicos (TNNP y otros): kernel CellML embebido, actualización de states, evaluación de corriente e integración local del ODE.
`applications/solvers/.../createFields.H`	Inicialización de campos y lectura de diccionarios.
`tutorials_electro/.../run_convergence.py`	Script driver.
`constant/cardiacProperties`	χ, C_m, $\mathbf{D}$, modelo iónico.
`constant/timeIntegrationProperties`	Solver de ODE, tolerancias, tope de subpasos.
`constant/spatialIntegrationProperties`	Esquema temporal, flags HO, opciones de JFNK / lineal / búsqueda de línea.
`constant/stimulusProtocol`	Regiones y temporización del estímulo.

C. Back-ends de álgebra lineal y solver hermano

Este solver *integra ambos back-ends* de álgebra lineal en un mismo ejecutable, elegibles en tiempo de ejecución con `linearSolverBackend` (ramas ensambladas) y `jfnkLinearSolverBackend` (Krylov de JFNK). El ensamblaje y el almacenamiento de matrices usan siempre `Eigen::SparseMatrix`; el back-end sólo determina cómo se resuelve el sistema.

Aspecto	`...Backend = Eigen`	`...Backend = PETSc`
Contenedor de matriz	`Eigen::SparseMatrix`	idem (se copia a `Mat`)
Solver directo disperso	`Eigen::SparseLU`	`KSPPREONLY+PCLU`
Solver iterativo (Picard/diag)	`Eigen::BiCGSTAB+ILUT`	`KSP+PC` (`gmres`+`ilu`, `cg`+`gamg`, ...)
Krylov interno de JFNK	GMRES propio	`KSPGMRES` sobre `MatShell`
Precondicionador de JFNK	`jfnkPreconditioner`: `none` o `diffusion` (ILU de $\mathbf{A}$) — disponible en ambos	

Los parámetros de convergencia (`nonlinearTolerance`, `jfnkEpsilon`, `jfnkLineSearch*`, `jfnkInitGuess*`,

`jfnkClampODEInput`, `nonlinearAcceptUnconverged`, `ODE_*`) tienen el mismo significado independientemente del back-end, de modo que la misma configuración de `run_convergence.py` produce una corrida algorítmicamente equivalente (salvo las diferencias de rendimiento del solve lineal).

Solver hermano `highOrderElectroActivationFoamImplicit`. Existe además una versión más simple, con back-end *solo Eigen* y sin las optimizaciones de la Sección E. Su diferencia algorítmica principal es que corre un número *fijo* de iteraciones no lineales (`nonlinearIterations`, por defecto 1) *sin* test de convergencia: con el valor por defecto es un esquema semi-implícito de *una* evaluación del ODE y *un* solve lineal por paso de tiempo. Esto lo hace mucho más rápido a costa de no controlar el residuo no lineal; a los pasos de tiempo pequeños del benchmark la diferencia en los resultados es marginal (ver Sección E y F).

D. Paralelización en memoria distribuida

Esta variante, `highOrderElectroActivationFoamImplicitPETScDistributed`, deja intactos todos los algoritmos de las secciones anteriores y reemplaza el álgebra lineal por una genuinamente distribuida. La distinción importa, porque hay tres solvers en la familia y solo el último es paralelo en sentido algebraico:

Variante	Álgebra lineal
`...ImplicitPETSc`	Malla serie, LRE serie, ensamblado Eigen, PETSc como solver sobre `PETSC_COMM_SELF`.
`...ImplicitPETScParallel`	Librería de alto orden paralela: la malla se descompone y los stencils cruzan de rank, pero el ensamblado sigue produciendo una matriz Eigen por rank y le entrega el CSR terminado a PETSc sobre `PETSC_COMM_SELF`. Serie por construcción.
`...ImplicitPETScDistributed`	Las matrices son `MPIAIJ` sobre `PETSC_COMM_WORLD` y todo producto matriz–vector pasa por `MatMult`.

Los tres se mantienen en paralelo a propósito: cada uno es la referencia contra la que se verifica el siguiente, en vez de compararse de memoria.

D.1. Disposición de la matriz: filas locales, columnas globales

Cada rank es dueño de las filas de sus propias celdas y de nada más. Las columnas, en cambio, son índices de celda *globales*, tomados del `globalIndex` que la librería de alto orden construye junto con sus stencils. Escribiendo n_{loc} para la cantidad de celdas de este rank y n_{glob} para el total, todo operador ensamblado tiene forma $n_{\text{loc}} \times n_{\text{glob}}$, y el desplazamiento de filas de este rank es

$$\texttt{gRowStart} \;=\; \texttt{globalIndex::localStart()}.$$

Esa es exactamente la disposición `MPIAIJ` de PETSc, así que la traducción del contenedor Eigen a la matriz distribuida (`buildDistributedMat`) solo tiene que desplazar el índice de fila y separar la prealocación en el bloque diagonal (columnas propias de este rank) y el extradiagonal. Acertar esa separación es lo que evita que el ensamblado degenere en realocaciones repetidas.

En serie `gRowStart` $= 0$ y $n_{\text{glob}} = n_{\text{loc}}$, de modo que el mismo camino de código produce exactamente la matriz secuencial que reemplaza.

Una familia de errores que conviene nombrar. Esa última propiedad es cómoda y peligrosa en la misma medida: *un índice local usado donde se espera uno global es invisible en serie.* Se encontraron cinco casos en esta familia de solvers, todos silenciosos con un solo rank:

Sitio	Síntoma con > 1 rank
Diagonal de la matriz de masa	No-ceros de más en $\mathbf{A}$, exactamente la cantidad de filas por rank.
Columna de `addDiagonalToMatrix`	Ídem.
Chequeo de tamaño en `updateValues`	Acepta un vector de largo equivocado.
Desplazamiento de fila en `updateValues`	Valores escritos en filas equivocadas.
Ensamblado ortogonal del hermano manufacturado	Columnas locales en todo el ensamblado, así que el operador no podía armarse correctamente.

La Sección D.4 explica por qué los chequeos habituales no ven ninguno de estos.

D.2. Qué tiene que cruzar una frontera de procesador

Tres clases de dato cruzan una partición, y cada una necesita un mecanismo distinto.

(i) Stencils de reconstrucción — nada que hacer. La librería de alto orden arma sus stencils geométricamente sobre un halo cuya profundidad controla `haloDepthScale`, y emite índices de columna *globales*. El término de flujo en una cara de procesador se ensambla entonces igual que en una cara interna: el stencil ya cruza el corte.

(ii) Datos centrados en celda — `syncTools`. La conductividad de la celda del otro lado de una cara acoplada, y su índice global, se obtienen con `syncTools::swapBoundaryCellList`, indexado por posición de cara de borde. Dos alternativas tentadoras son incorrectas:

- un *campo* de patch de procesador contiene el valor interpolado *en la cara*, no el valor de la *celda* vecina;
- `mesh.C()` es un `slicedVolVectorField` cuyos valores acoplados solo son válidos después de una evaluación que el ensamblado no realiza.

Por la misma razón el vector propietario–vecino $\mathbf{d}_{PN}$ se toma de `patch.delta()`, que para un `processorFvPatch` devuelve $\mathbf{C}_N - \mathbf{C}_P$ construido con la geometría de ambos lados, y nunca de `C[nei] - C[own]`.

(iii) Filas de matriz de largo variable — `PstreamBuffers`. El salto de estabilización es el único término que necesita la fila de reconstrucción *completa* de la celda vecina: hasta ~ 60 pares (columna global, coeficiente) para un stencil $p3$ en 3-D. Ningún intercambio de campos puede transportar eso.

El mecanismo, implementado en el hermano manufacturado `highOrderManufacturedFDAImplicitPETScDistribut` es que el rank *dueño* de la celda arme la fila él mismo y envíe los pares, evaluados en el centro de la cara — un punto que ambos ranks tienen. No se intercambia nada más: ni centros de celda, ni stencils, ni tablas de coeficientes. Escribir el salto desde el punto de vista de la celda local,

$$\frac{a}{V_c}\Big(T_{\text{lejana}}(\mathbf{x}_f) - T_c(\mathbf{x}_f)\Big),$$

donde T_c es la reconstrucción de Taylor de la celda c, elimina además la convención propietario/vecino: el $+$ que el lazo de caras internas escribe en la fila del propietario y el $-$ que escribe en la del vecino son la misma expresión vista desde cada celda, y el coeficiente de cara a depende de la orientación solo a través de $|\mathbf{d}_{PN}\cdot\mathbf{n}|$ y $\mathbf{n}\cdot\mathbf{D}\cdot\mathbf{n}$. Ambos ranks calculan entonces el mismo a sin ponerse de acuerdo en nada.

Estado en este solver. La rama acoplada del ensamblado ortogonal agrega el término de flujo y *no* el salto de Rhie–Chow. Eso es inocuo con el $\alpha = 0$ por defecto que usan los benchmarks, y está documentado en el fuente en vez de omitido en silencio; portar el intercambio descrito arriba es el punto pendiente.

D.3. Reducciones globales

Todo lo que antes era un lazo sobre todas las celdas y ahora es un lazo sobre las celdas de este rank necesita una reducción: las normas de error de la Sección 14 (`gSum` sobre volúmenes y errores al cuadrado), los residuos no lineales y sus tests de parada, y el muestreo de tiempos de activación.

La que es fácil pasar por alto es la *búsqueda de línea de Armijo*, condición (36). El paso aceptado tiene que ser un solo número para todo el dominio; sin las reducciones dentro de la búsqueda, cada rank aceptaría un λ distinto y el iterado de Newton dejaría de ser función únicamente de la malla. Que la corrida paralela reproduzca *exactamente* el `lineSearchIters` de la serie es lo que valida esto.

D.4. Cómo se verifica un operador paralelo

Dos chequeos son baratos y *no* alcanzan por sí solos:

- **Sumas de fila.** Con bordes de flujo nulo un campo constante tiene gradiente nulo, así que toda fila de **K** debe sumar cero. Esto detecta un operador vacío o mal formado — detectó un `CompactListList::operator[]` que devuelve una vista por valor, que compilaba pero producía un stencil vacío — pero no puede ver una cara *omitida*, porque omitir una cara no aporta nada a ninguna de sus dos filas y las sumas siguen anulándose.
- **Huella del operador** (nnz global, $\sum a_{ij}$, $\sum |a_{ij}|$, $\max_i |\sum_j a_{ij}|$). Como los stencils son geométricos, el nnz global debe coincidir exactamente con el valor serie. Pero un valor escrito en la *columna equivocada* no cambia ni el nnz, ni ninguna suma, ni ninguna suma de fila.

El chequeo que sí decide es una comparación entrada por entrada. Con `CF_DUMP_K` definida en el entorno, el hermano manufacturado escribe **K** como (fila global, columna global, valor), un archivo por rank; mapear los índices paralelos de vuelta con `processor`N`/constant/polyMesh/cellProcAddressing` da un diff directo contra el ensamblado serie. `CF_DUMP_STENCILS` hace lo mismo un nivel más abajo, para la *membresía* de los stencils y los centros de celda, que es lo que distingue un coeficiente distinto de un stencil distinto.

Comparar iteración por iteración. Usar `petscLinearPcType jacobi`. PETSc envuelve `ilu` y `lu` en block-Jacobi en paralelo, así que esos precondicionadores dependen de la partición *por construcción*: con `ilu` un caso manufacturado convergido muestra una desviación de $4{,}5 \times 10^{-6}$ entre uno y dos ranks que es enteramente algebraica, y con `jacobi` el mismo caso coincide en todos los dígitos impresos. Confundir eso con un error de discretización cuesta mucho tiempo.

D.5. Dependencia de la partición en los stencils MLS

Nota (2026-10). En el solver MMS se encontró después que buena parte de esta discrepancia venía del caso y no de la librería: la clave `Nn` son las celdas que se *agregan* sobre el mínimo del grado, y los barridos la escribían como si fuera el total, así que los stencils eran un 50 % más grandes de lo pretendido y el halo no alcanzaba a cubrirlos. Con el tamaño correcto, en 2-D, M11 mide 0 stencils distintos entre np=1 y np=2 y M15 baja a 7×10^{-15}. Las mediciones 3-D de abajo son anteriores a ese arreglo y no se volvieron a correr.

Queda una inconsistencia paralela genuina, y vive en la selección de stencils de la librería de alto orden, no en ningún solver.

Mecanismo. Un stencil pretende ser *las N celdas cuyos centros están más cerca del centro de la cara*. La librería llega a eso en dos pasos: primero arma un conjunto de candidatas creciendo capas completas de `cellCells` hasta que el conjunto tiene $\lceil 1{,}5N \rceil$ celdas, y después se queda con las N más cercanas por distancia euclídea. El segundo paso es geométrico; **el primero es topológico**. En una malla de tetraedros "dentro de k saltos" y "dentro de un radio r" no son el mismo conjunto, y con apenas $0{,}5N$ de holgura las últimas posiciones del stencil las decide la conectividad y no la geometría.

Al descomponer, el crecimiento por capas se frena en el corte, y el otro lado lo aporta en cambio una consulta que devuelve todas las celdas del rank vecino dentro de una caja envolvente — un conjunto *geométrico*, y generoso. Cerca de una partición la selección mezcla entonces un conjunto de candidatas topológico de un lado con uno geométrico del otro. Por eso la desviación no responde a `haloDepthScale`: la caja nunca fue la restricción activa.

Mediciones (tetraedros no estructurados 3-D, 4971 celdas, un rank contra dos, $\alpha = 0$ para que no intervenga la estabilización):

Observable	Valor
Stencils de cara que difieren	536 de 10676
Su tamaño	idéntico (80): los miembros se *sustituyen*, no se pierden
Posición de los miembros intercambiados	98 % en el último 10 % del stencil ordenado por distancia
Filas de $\mathbf{K}$ que difieren	552 de 4971, hasta 4.17 contra $\max\lvert K\rvert = 256$
Caras de corte cuyos dos ranks discrepan *entre sí*	9
Centros de celda, serie vs descompuesta	hasta $3{,}3 \times 10^{-16}$ (la suma del centroide recorre una lista de caras renumerada)

Las nueve caras de corte son un defecto por derecho propio, sin necesidad de comparar con la serie: las dos filas de una misma cara quedan armadas con stencils de flujo distintos, así que el flujo que sale de una celda no es el que entra a su vecina y el esquema no es conservativo ahí.

Cuánto importa. Comparando los *campos* de V_m celda por celda en vez de la desviación de una norma, con $\alpha = 0{,}1$ y `jacobi`:

$$\frac{\left\| V_\mathrm{m}^{np=1} - V_\mathrm{m}^{np=2} \right\|}{\| V_\mathrm{m} - V_\mathrm{m}^{\mathrm{exacta}} \|} = \frac{1{,}51 \times 10^{-5}}{6{,}84 \times 10^{-4}} = 2{,}2 \times 10^{-2}, \qquad \frac{\|\Delta V_\mathrm{m}\|}{\text{amplitud}} = 7{,}5 \times 10^{-6}.$$

Es decir, la diferencia entre particiones es un 2,2 % del error de discretización.

En un barrido de refinamiento $N = 8, 10, 12, 14$ la desviación se mantiene en el nivel 10^{-4}–10^{-3} mientras el error mismo cae de $1{,}19 \times 10^{-3}$ a $2{,}28 \times 10^{-4}$: la perturbación se achica al mismo ritmo que el error. En consecuencia el orden de convergencia ajustado queda intacto,

$$p_{V_\mathrm{m},L_2}^{np=1} = 2{,}90722821, \qquad p_{V_\mathrm{m},L_2}^{np=2} = 2{,}90722635,$$

una diferencia de $1{,}9 \times 10^{-6}$ contra una tolerancia de aceptación de 0,05. Eso es lo que predicen dos esquemas *consistentes* del mismo orden formal: su diferencia es una perturbación del mismo orden, no un error sistemático.

Cómo se arreglaría.

1. Hacer geométrico el conjunto de candidatas de los dos lados — consultar centros de celda por radio (`indexedOctree`) en vez de crecer capas de `cellCells`, con el radio tomado de la estimación por capas como cota superior. El stencil pasa a ser función de (centro de cara, centros de celda, N) y nada más, invariante ante la partición.

2. Frenar el recorte de empates: la rama paralela filtra candidatas por radio *antes* de que corra la expansión por distancia igual, así que un grupo de empate que cruza ese radio queda partido al medio (17 miembros en serie contra 16 con dos ranks, en un caso estructurado 2-D). Inflar el radio por $(1 + \texttt{relTol})$ y hacer pasar el retorno temprano de la serie por el mismo código de selección lo cierra.

3. Afirmar completitud en vez de fallar en silencio: verificar que la N-ésima distancia queda estrictamente dentro del radio consultado.

4. Una cara, un stencil: que el lado de rank menor de una cara de corte sea dueño del stencil y lo envíe, lo que restaura la conservación a través del corte.

`relTol` (10^{-8}), `overSampleFactor` (1,5) y `searchExpansionFactor` (1,3) son argumentos por defecto hard-codeados, no entradas de diccionario, así que nada de esto se puede explorar desde un caso: es un cambio de librería.

Una advertencia antes de tocarlo. La corrida en serie es la truncada — su conjunto de candidatas es puramente topológico en todas partes, no solo cerca de un corte — así que arreglar la selección cambia también los resultados en serie, y con ellos toda tabla de convergencia producida hasta ahora. Esa es una decisión a tomar deliberadamente, no un efecto colateral a absorber.

D.6. Casos de verificación: qué se corrió y dónde está

Todos los casos están bajo `tutorials_electro/electroTestParallelVerification/`. El driver es `tutorials_electro/electroDistValidation/`; los directorios de acá son copias suyas con un parámetro cambiado cada una.

Directorio	Prueba	Qué establece
`electroA3_np1`, `_np2`, `_np4`	A3, aceptación física	tiempo de activación en P8 y ve dad de conducción
`electroA4_np2`, `_np4`	A4, invariancia a la descomposición	`simple` en vez de `scotch`: otra metría de corte
`electroTri_np1`, `_np2`, `_np4`	malla no estructurada	la ruta de gmsh, 1500 celdas
`matrix_el_2D_np1`, `_np2`, `matrix_el_3D_*`	matriz de métodos	Picard / JFNK / diagonalIion, + triangular, 2-D y 3-D
`jac_2D_np1`, `jac_2D_np2`	aislar el preacondicionador	`jacobi`, el único independiente número de ranks
`perf_{ilu,jac,gamg,hypre}_np{1,2}`	rendimiento del solver lineal	$\Delta x = 0{,}5\,\mathrm{mm}$, 640 celdas
`perf025_{ilu,jac,gamg,hypre}_np{1,2}`	ídem, refinado	$\Delta x = 0{,}25\,\mathrm{mm}$, 2560 celdas
`pre_{0p0,0p1}_np{1,2}`	estabilización, antes	con el binario `electroPreStab`
`post_{0p0,0p1}_np{1,2}`	estabilización, después	con el binario porteado
`stab_{0p0,0p1}_np{1,2}`	primer intento de línea base	*descartado*: hubo recompilacione rante el barrido

A3, aceptación física (`electroA3_*`). Hexa, $\Delta x = 0{,}5\,\mathrm{mm}$, $\Delta t = 10^{-4}$, tríada $(p3, \mathrm{CCp3}, p3)$, corrida completa hasta que el frente alcanza P8. Corridas secuenciales con la máquina ociosa, de modo que los tiempos de pared son válidos.

Ranks	Estado	T_{P8} [ms]	desv.	CV [m/s]	Wall [s]	Speed-up
1	OK	52.0795930112	—	0.38388	310.3	1,00×
2	OK	52.0795930123	$2{,}11 \times 10^{-11}$	0.38388	177.0	1,75×
4	OK	52.0795930133	$4{,}03 \times 10^{-11}$	0.38388	102.9	3,01×

`OK` es la entrada más importante de cada fila: es el detector binario de bloqueo de conducción, y dice que el frente cruza todos los cortes de partición. La tolerancia del 0,5 % propuesta para $|\Delta T_{P8}|/T_{P8}$ estaba *equivocada por ocho órdenes de magnitud* — el spread observado es 4×10^{-11}. Picard converge al mismo punto fijo de forma determinista, así que un residuo de corte alto no introduce deriva.

A4, invariancia a la descomposición (`electroA4_*`). La misma corrida cortada con `simple` ($n = (2\,1\,1)$ y $(2\,2\,1)$) en vez de `scotch`: `OK` en ambas, T_{P8} dentro de $9{,}22 \times 10^{-11}$ de la serie, CV idéntica a cinco decimales. Que dos geometrías de corte no relacionadas coincidan es evidencia mucho más fuerte que varios conteos de ranks bajo un solo método.

Malla no estructurada (`electroTri_*`). 1500 celdas por la ruta de gmsh: `OK` con 1, 2 y 4 ranks, T_{P8} dentro de $6{,}55 \times 10^{-11}$, speed-up 1,65× y 2,84×.

Rendimiento del solver lineal (`perf_*`, `perf025_*`). Las iteraciones de Krylov por iteración no lineal varían más de un factor tres entre preacondicionadores mientras los tiempos de pared coinciden dentro de un 8 %, en los dos tamaños de malla. *El solve lineal no es el cuello de botella* — lo son las ODEs iónicas — así que cambiar de preacondicionador no compra nada acá. Refinar de 0,5 a 0,25 mm multiplica el costo por 4,3, o sea esencialmente lineal en el número de celdas, y los conteos de iteraciones se mantienen planos (ilu 3.0, hypre 4.0, jacobi 8.0), así que no aparece ningún cruce. `gamg` falló en la malla fina y no se diagnosticó.

Aislar el preacondicionador (`jac_2D_*`). `ilu` no tiene implementación MPIAIJ y en paralelo se envuelve en block-Jacobi, que depende de la partición por construcción; `jacobi` es puramente diagonal y por lo tanto el mismo operador con cualquier número de ranks. Al cambiarlo colapsaron las diferencias de trayectoria entre $np = 1$ y $np = 2$: cinco de seis configuraciones pasaron a ser *idénticas*, y los conteos de filas coincidieron en las seis (`triangular` con Picard pasó de 184 contra 275 filas a 184 contra 184).

Estabilización en caras de procesador (`pre_*` contra `post_*`). Desviación del trazo de residuos entre $np = 1$ y $np = 2$ con $\alpha = 0{,}1$:

Malla	Método	Antes	Después	Control $\alpha = 0$	Mejora
hexa	Picard	$3{,}23 \times 10^{-9}$	$9{,}06 \times 10^{-12}$	$8{,}53 \times 10^{-12}$	356×
hexa	JFNK	$2{,}58 \times 10^{-5}$	$2{,}59 \times 10^{-5}$	$2{,}48 \times 10^{-5}$	1×
hexa	diagonalIion	$1{,}27 \times 10^{-8}$	$4{,}51 \times 10^{-10}$	$2{,}28 \times 10^{-10}$	28×
triangular	Picard	$2{,}45 \times 10^{-7}$	$4{,}68 \times 10^{-11}$	$9{,}55 \times 10^{-12}$	5240×
triangular	JFNK	$4{,}44 \times 10^{-6}$	$4{,}43 \times 10^{-6}$	$4{,}64 \times 10^{-6}$	1×
triangular	diagonalIion	$1{,}59 \times 10^{-7}$	$6{,}17 \times 10^{-9}$	$5{,}59 \times 10^{-10}$	26×

La salida serial es bit-idéntica (SHA-256) en las doce configuraciones, con los dos valores de α. Los caminos ensamblados colapsan sobre el control de $\alpha = 0$, que es el piso alcanzable. JFNK no se mueve, y es esperable: su desviación la domina su propio preacondicionador — el ILUT de Eigen sobre el bloque diagonal, dependiente de la partición por construcción — y no la estabilización del ensamblado. El mismo $2{,}48 \times 10^{-5}$ aparece con `ilu` y con `jacobi`, que es lo que lo identificó.

Para esta comparación se construyeron dos binarios *distintos*, `electroPreStab` y el solver porteado, porque un primer intento (`stab_*`) tuvo recompilaciones cayendo mientras el barrido corría. Eso importa: si el "antes" ya hubiera tenido la corrección, el "después" habría salido idéntico, y un resultado nulo es indistinguible de uno que funciona.

E. Optimizaciones de rendimiento

Esta sección documenta cuatro optimizaciones incorporadas al solver. Todas son *aditivas* y están gobernadas por claves de diccionario cuyo valor por defecto reproduce **exactamente** el comportamiento previo: ninguna corrida existente cambia de resultado salvo que se activen explícitamente. Actualizan además algunas filas de la tabla comparativa de la Sección anterior (el back-end ensamblado ahora reutiliza la factorización, el JFNK admite un precondicionador, y el ODE iónico puede integrarse en paralelo con OpenMP).

E.1. Reutilización de la matriz y la factorización (Fase B)

El operador implícito $\mathbf{A} = \mathbf{M}/\Delta t - \theta\,\mathbf{L}$ depende sólo de las matrices constantes $\mathbf{M}$, $\mathbf{L}$, del parámetro θ y del paso Δt. Como Δt es fijo salvo en el último paso recortado, $\mathbf{A}$ es constante durante casi toda la simulación. Antes, $\mathbf{A}$ se reensamblaba y **se re-factorizaba** en cada iteración no lineal; ahora se ensambla una sola vez y la factorización (KSP de PETSc, o `Eigen::SparseLU/BiCGSTAB`) se conserva durante todo el run. Una política de actualización distingue tres casos por solve:

- **Rebuild**: primer uso o cambio de Δt ⇒ setup completo (asignación + factorización simbólica + numérica).
- **ValuesOnly**: mismo patrón de dispersidad, valores nuevos (la diagonal cambia cada iteración en `diagonalIion`) ⇒ se refrescan los valores reutilizando el patrón (`updateValues`).
- **Reuse**: matriz idéntica al solve anterior (Picard con operador constante) ⇒ no se toca la matriz, sólo se resuelve.

La clave `petscReusePreconditioner` (por defecto `false`) permite además, en modo `diagonalIion`, reutilizar el precondicionador entre iteraciones internas cuando el cambio de diagonal es pequeño. En Picard esta optimización es puramente computacional: se resuelve la misma matriz con la misma factorización, por lo que los resultados son idénticos bit a bit (Sección F.1).

E.2. Precondicionador de difusión para el JFNK (A3)

El Jacobiano libre de matriz del JFNK se resolvía sin precondicionador, de modo que cada iteración de Krylov requiere una evaluación completa del residuo (y por lo tanto una integración del ODE iónico). Se agrega un precondicionador *por la derecha* $\mathbf{M}^{-1}$ con $\mathbf{M} = \mathbf{A}$, la parte lineal (difusiva, SPD) del Jacobiano. Para evitar el relleno catastrófico de una factorización LU exacta del operador de alto orden (de esténcil ancho), se usa una **factorización LU incompleta** (`Eigen::IncompleteLUT`, ILUT), que se calcula una sola vez y se reutiliza; cada paso de Krylov cuesta entonces un solo *triangular solve* barato. Es muy efectiva aquí porque, con Δt pequeño, $\mathbf{A} \approx \mathbf{M}/\Delta t$ es fuertemente diagonal-dominante. La clave `jfnkPreconditioner` toma los valores `none` (por defecto, camino legado) o `diffusion`. En la rama PETSc se fija `KSPSetPCSide(PC_RIGHT)` para que el KSP monitoree el residuo verdadero $\|b - Ax\|$; en la rama del GMRES propio (Eigen) el precondicionador se aplica en el paso de Arnoldi y en la reconstrucción de la corrección. No cambia la solución convergida, sólo el camino de convergencia (menos matvecs, es decir, menos integraciones del ODE).

E.3. Integración del ODE iónico en paralelo (C1)

El bucle sobre los puntos de integración de `TNNP::solveODE` es *embarrassingly parallel* (cada punto es un ODE independiente). Se paraleliza opcionalmente con OpenMP mediante la clave `parallelODE` (por defecto `false`) leída del diccionario del modelo iónico. Como la clase base `ionicModel` comparte un único `ODESolver` (con scratch interno de las etapas de Runge–Kutta), se agrega un *pool* de solvers por hilo (`ensureOdeSolverPool/odeSolver(tid)`); el índice de diagnóstico `activeIntegrationPoint_` pasa a ser por hilo y la escritura del registro de protección se serializa con `#pragma omp critical`. Fue necesario agregar `-fopenmp` a las opciones de compilación de la biblioteca `ionicModels`. El número de hilos se controla con la variable de entorno `OMP_NUM_THREADS`; `run_convergence.py` lo expone mediante las constantes `PARALLEL_ODE` y `ODE_NUM_THREADS`. Los resultados son numéricamente idénticos a la versión serial (validado bit a bit).

E.4. Precondicionadores AMG (C3)

La ruta KSP ya soportaba pasar opciones de PETSc en tiempo de ejecución, de modo que los precondicionadores de multigrid algebraico (`gamg` nativo, `hypre` BoomerAMG) son alcanzables directamente. Se agregan los alias `amg`→`gamg` y `boomeramg`→`hypre` en el mapeo de `petscLinearPcType`. Para el operador SPD del monodominio la combinación teóricamente recomendable en mallas 3D grandes es `cg`+`gamg`; sin embargo, los experimentos (Sección F.5) muestran que para los pasos de tiempo usados aquí el sistema está dominado por la masa (bien condicionado) y AMG no aporta.

Resumen de claves nuevas. Todas con valor por defecto igual al comportamiento previo.

Clave	Defecto	Efecto
`petscReusePreconditioner`	`false`	Reutiliza el precondicionador entre iteraciones en `diagonalIion`.
`jfnkPreconditioner`	`none`	`diffusion`: precondiciona el Jacobiano JFNK con ILUT de $\mathbf{A}$.
`parallelODE`	`false`	Integra los ODE iónicos con OpenMP (dic. del modelo iónico).
`petscLinearPcType`	`ilu`	Ahora acepta `gamg`/`hypre` (y alias `amg`/`boomeramg`).

F. Experimentos de rendimiento

Se midieron las optimizaciones anteriores en el benchmark 2D estilo Niederer (malla de 23 726 celdas salvo indicación) con el modelo TNNP e integrador RKF45, esquema Crank–Nicolson y masa consistente en los casos de alto orden. Salvo aclaración, el estímulo es el del benchmark (una esquina de 1,5 mm). Las corridas son cortas (pocos pasos) y las mediciones se tomaron con `/usr/bin/time -v`; cuando corresponde se reporta `loopWall` (tiempo del bucle temporal, del `solverPerformance.dat`) para aislar el costo por paso del setup único. La máquina tiene 20 núcleos.

F.1. Fase B: idéntico y más rápido en Picard

Se compararon las mismas corridas del barrido de convergencia contra un binario *previo* a los cambios (Picard, Crank–Nicolson, masa consistente, RKF45, back-end PETSc `gmres`+`ilu`), tomando los cuatro casos presentes en ambas carpetas.

Δt / Δx	P8 [ms] (nuevo=viejo)	iter. no lin.	iter. lin.	wall nuevo/viejo [s]	Δ
10^{-4} / 0,25 mm	62.9948077742	3510=3510	12748=12748	1488.7 / 1645.3	−9,5 %
10^{-4} / 0,5 mm	153.3156018900	6293=6293	18879=18879	684.0 / 752.0	−9,0 %
10^{-5} / 0,25 mm	67.2950937434	6854=6854	13708=13708	2531.9 / 2786.9	−9,2 %
10^{-5} / 0,5 mm	157.1464148850	16136=16136	32272=32272	1472.6 / 1616.7	−8,9 %

Conclusión. El tiempo de activación de P8 coincide a diez decimales ($\Delta = 0$) y el número de iteraciones no lineales y lineales es idéntico, mientras que el wall time baja $\sim 9\,\%$. Como el uso de CPU fue del 99 % (serial), el ahorro proviene íntegramente de no re-factorizar la matriz constante, no de paralelismo.

F.2. A3: precondicionador de difusión en el JFNK

Mismo caso (esquina, 10 pasos, JFNK, back-end PETSc), variando `jfnkPreconditioner`.

`jfnkPreconditioner`	iter. de Krylov (total)	wall [s]
`none`	90	98.6
`diffusion`	24	59.0

Conclusión. El precondicionador reduce los matvecs de 90 a 24 (3,75× menos integraciones del ODE) y el wall time 1,67× (98,6 → 59,0 s), con la misma solución (Vm coincidente dentro de la tolerancia de Krylov). Precondicionar es claramente más rápido que no hacerlo.

F.3. C1: paralelización del ODE (no ayuda en este problema)

Test A/B con `OMP_NUM_THREADS` fijo en 8, alternando `parallelODE` (la diferencia aísla el efecto de C1, ya que el threading interno de PETSc/BLAS es el mismo en ambos). Para los casos de alto orden y 3D se reporta `loopWall`.

Configuración	`parallelODE=true`	`=false`	¿ayuda?
Picard bajo orden (2D)	46.9	46.9	no
JFNK + `diffusion` (2D)	57.06	57.58	no
JFNK + `none` (2D)	28.76	28.96	no
`gaussPointODE` + HO p3 (2D, loop)	29.259	29.262	no
3D, 17 280 celdas (loop)	2.861	2.816	no

Se probaron específicamente los dos escenarios donde cabría esperar beneficio: `gaussPointODE` con alto orden p3 (varios ODE por celda, $\sim$16$\times$ más trabajo iónico) y 3D con muchas celdas. **Conclusión.** En todos los casos el resultado es idéntico entre `true`/`false` (y el % de CPU no cambia), es decir, la región paralela del ODE aporta ~ 0. El motivo es que el ODE iónico (RKF45 sobre TNNP, sobre todo en reposo) es una *fracción pequeña* del costo: dominan la reconstrucción LRE en los puntos de Gauss, el promediado de I_{ion} y el ensamblaje/solve de alto orden, que son operaciones seriales que crecen igual o más al agrandar el problema. C1 es funcionalmente correcto (resultados bit-idénticos) pero no acelera este perfil de problema; sólo rendiría con modelos mucho más rígidos o frentes de activación extensos.

F.4. Threading interno de PETSc/BLAS (contraproducente en Picard)

Picard, `parallelODE`=`false`, variando `OMP_NUM_THREADS` (20 pasos, esquina).

`OMP_NUM_THREADS`	wall [s]	CPU	User [s]
1	45.7	98 %	44.9
20	47.3	197 %	68.8

Conclusión. Con más hilos aparece paralelismo interno de PETSc/BLAS/Eigen ($\sim 2\times$ de CPU), pero el wall time no baja (sube levemente): es *memory-bound* y sólo desperdicia CPU. Conviene correr Picard con `OMP_NUM_THREADS=1`.

F.5. Solver lineal: AMG vs. ILU (el solve no es el cuello)

Comparación de `petscLinearKspType`/`petscLinearPcType` con `OMP_NUM_THREADS=1`.

Caso	Solver lineal	`loopWall` [s]	iter. lin. (total)
3D, 17 280, Δt=10^{-6}, bajo orden	`gmres+ilu`	5.27	16
	`cg+gamg`	5.72	24
2D, 23 726, Δt=10^{-4}, HO consistente	`gmres+ilu`	70.21	40
	`gmres+gamg`	70.66	72
	`cg+gamg`	70.47	72

Conclusión. AMG es levemente más lento y usa *más* iteraciones: con Δt pequeño el sistema está dominado por la masa ($\mathbf{A} \approx \mathbf{M}/\Delta t$), bien condicionado y de tipo reacción, donde ILU ya es un excelente precondicionador local y la jerarquía de multigrid es contraproducente. Más importante aún: el `loopWall` es prácticamente el mismo ($\sim$ 70 s) sea cual sea el solver lineal (40 vs. 72 iteraciones), lo que demuestra que **el solve lineal no es el cuello de botella**. Junto con los experimentos anteriores, el costo dominante en el régimen Picard de alto orden es la maquinaria de alto orden en sí (reconstrucción LRE y manejo de puntos de Gauss), no el ODE ni el solver lineal.

F.6. Síntesis y recomendaciones

- Mantener `parallelODE`=`false` y `OMP_NUM_THREADS=1`: ni C1 ni el threading interno aceleran este problema.
- Mantener `petscLinearPcType`=`ilu` (no usar AMG): el sistema está bien condicionado y el solve lineal no es el cuello.
- Las ganancias reales, sin paralelismo, son la Fase B (reutilización de factorización, $\sim 9\,\%$ en Picard, resultados idénticos) y A3 (`jfnkPreconditioner`=`diffusion`, 1,67× en JFNK).
- El objetivo natural para acelerar el régimen de alto orden sería paralelizar o abaratar la reconstrucción LRE y el manejo de los puntos de Gauss (no realizado aquí).

Referencias

[1] K. H. W. J. ten Tusscher, D. Noble, P. J. Noble, A. V. Panfilov, *A model for human ventricular tissue*, AJP – Heart and Circulatory Physiology **286** (2004) H1573–H1589.

[2] S. A. Niederer et al., *Verification of cardiac tissue electrophysiology simulators using an N-version benchmark*, Prog. Biophys. Mol. Biol. **104** (2012) 22–48.

[3] D. A. Knoll, D. E. Keyes, *Jacobian-Free Newton-Krylov methods: a survey of approaches and applications*, J. Comp. Phys. **193** (2004) 357–397.

[4] J. Nocedal, S. J. Wright, *Numerical Optimization*, 2nd edition, Springer, 2006.

[5] Y. Saad, *Iterative Methods for Sparse Linear Systems*, 2nd edition, SIAM, 2003.

[6] E. Hairer, G. Wanner, *Solving Ordinary Differential Equations II*, Springer, 1996.